你說：

write sklearn to solve 50-startup multiple linear regression problem, following crisp-dm

ChatGPT 說：

Below is a complete **scikit-learn workflow for the 50_Startups dataset**, organized by **CRISP-DM** and using **Multiple Linear Regression**. I'm assuming the usual columns: `R&D Spend`, `Administration`, `Marketing Spend`, `State`, and target `Profit`.

CRISP-DM flow

1. **Business Understanding** — predict startup profit.
2. **Data Understanding** — inspect shape, columns, types, missing values, statistics.
3. **Data Preparation** — separate X/y, one-hot encode `State`, train/test split.
4. **Modeling** — fit `LinearRegression`.
5. **Evaluation** — MAE, MSE, RMSE, R^2, coefficients, actual vs predicted.
6. **Deployment / Interpretation** — use the trained pipeline for new startups.

```
# ============================================================
# 50 STARTUPS - MULTIPLE LINEAR REGRESSION
# Following CRISP-DM
# ============================================================

# ============================================================
# 0. IMPORT LIBRARIES
# ============================================================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LinearRegression
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)
```

```
# ============================================================
# 1. BUSINESS UNDERSTANDING
# ============================================================

"""
Goal:
Predict the Profit of a startup using:

1. R&D Spend
2. Administration
3. Marketing Spend
4. State

Target variable:
Profit

Problem type:
Supervised Learning - Regression

Model:
Multiple Linear Regression
"""



# ============================================================
# 2. DATA UNDERSTANDING
# ============================================================

# Load dataset
df = pd.read_csv("50_Startups.csv")

# Show first 5 rows
print("First 5 rows:")
print(df.head())

print("\nDataset shape:")
print(df.shape)

print("\nColumn names:")
print(df.columns)

print("\nData types:")
print(df.dtypes)

print("\nDataset information:")
```

```
df.info()

print("\nMissing values:")
print(df.isnull().sum())

print("\nDescriptive statistics:")
print(df.describe())


# ============================================================
# 3. DATA PREPARATION
# ============================================================

# -------------------------
# 3.1 Define X and y
# -------------------------

X = df.drop("Profit", axis=1)
y = df["Profit"]

print("\nFeatures:")
print(X.head())

print("\nTarget:")
print(y.head())


# -------------------------
# 3.2 Identify columns
# -------------------------

numeric_features = [
    "R&D Spend",
    "Administration",
    "Marketing Spend"
]

categorical_features = [
    "State"
]


# -------------------------
# 3.3 Preprocessing
# -------------------------
```

```
# One-hot encode State.
#
# drop='first' removes one dummy category.
# This avoids perfect dummy-variable multicollinearity.

preprocessor = ColumnTransformer(
    transformers=[
        (
            "categorical",
            OneHotEncoder(drop="first", handle_unknown="ignore"),
            categorical_features
        )
    ],
    remainder="passthrough"
)



# -------------------------
# 3.4 Train / Test Split
# -------------------------

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

print("\nTraining samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])



# ============================================================
# 4. MODELING
# ============================================================

# Build pipeline
model = Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        ("regressor", LinearRegression())
    ]
)
```

```
# Train model
model.fit(X_train, y_train)

print("\nModel training completed.")


# ============================================================
# 5. EVALUATION
# ============================================================

# Predict test data
y_pred = model.predict(X_test)


# -------------------------
# 5.1 Actual vs Predicted
# -------------------------

results = pd.DataFrame({
    "Actual Profit": y_test.values,
    "Predicted Profit": y_pred
})

print("\nActual vs Predicted:")
print(results)


# -------------------------
# 5.2 Evaluation Metrics
# -------------------------

mae = mean_absolute_error(y_test, y_pred)

mse = mean_squared_error(y_test, y_pred)

rmse = np.sqrt(mse)

r2 = r2_score(y_test, y_pred)

print("\n========== MODEL EVALUATION ==========")

print(f"MAE  : {mae:.2f}")
print(f"MSE  : {mse:.2f}")
print(f"RMSE : {rmse:.2f}")
print(f"R²   : {r2:.4f}")
```

```
# -------------------------
# 5.3 Training R²
# -------------------------

train_r2 = model.score(X_train, y_train)

print(f"Training R² : {train_r2:.4f}")
print(f"Testing R²  : {r2:.4f}")


# ============================================================
# 5.4 MODEL COEFFICIENTS
# ============================================================

# Get feature names after preprocessing

feature_names = model.named_steps[
    "preprocessor"
].get_feature_names_out()

coefficients = model.named_steps[
    "regressor"
].coef_

coef_df = pd.DataFrame({
    "Feature": feature_names,
    "Coefficient": coefficients
})

coef_df["Absolute Coefficient"] = abs(
    coef_df["Coefficient"]
)

coef_df = coef_df.sort_values(
    "Absolute Coefficient",
    ascending=False
)

print("\n========== COEFFICIENTS ==========")
print(coef_df)


# Intercept
```

```
intercept = model.named_steps[
    "regressor"
].intercept_

print("\nIntercept:")
print(intercept)


# ============================================================
# 5.5 ACTUAL VS PREDICTED GRAPH
# ============================================================

plt.figure(figsize=(7, 5))

plt.scatter(
    y_test,
    y_pred
)

# Perfect prediction reference line

minimum = min(y_test.min(), y_pred.min())
maximum = max(y_test.max(), y_pred.max())

plt.plot(
    [minimum, maximum],
    [minimum, maximum]
)

plt.xlabel("Actual Profit")
plt.ylabel("Predicted Profit")
plt.title("Actual vs Predicted Profit")

plt.show()


# ============================================================
# 5.6 RESIDUAL ANALYSIS
# ============================================================

residuals = y_test.values - y_pred

plt.figure(figsize=(7, 5))
```

```
plt.scatter(
    y_pred,
    residuals
)

plt.axhline(
    y=0
)

plt.xlabel("Predicted Profit")
plt.ylabel("Residual")
plt.title("Residual Plot")

plt.show()


# ============================================================
# 6. DEPLOYMENT / NEW DATA PREDICTION
# ============================================================

new_startup = pd.DataFrame({
    "R&D Spend": [150000],
    "Administration": [120000],
    "Marketing Spend": [300000],
    "State": ["California"]
})

new_prediction = model.predict(new_startup)

print("\n========== NEW STARTUP ==========")

print(new_startup)

print(
    "\nPredicted Profit:",
    round(new_prediction[0], 2)
)
```

For your report, you can describe the model as:

> Multiple Linear Regression was used to examine the relationship between startup expenditure, location, and profit. The dependent variable was `Profit`, while `R&D Spend`, `Administration`, `Marketing Spend`, and `State` were used as independent variables. The categorical variable

`State` was transformed using one-hot encoding, and the dataset was divided into 80% training data and 20% testing data with `random_state=42`. Model performance was evaluated using MAE, MSE, RMSE, and R^2.

One important point for this dataset: **feature scaling is not required for ordinary `LinearRegression`**. The coefficients can be estimated directly, although scaling would become more relevant for models such as Ridge or Lasso.

Also, because the dataset has only **50 observations**, a single 80/20 split leaves only 10 test observations. For an assignment, I'd recommend adding **feature selection and cross-validation** next, especially if your HW requirement specifically says feature selection is mandatory.

你說：



剛剛的程式碼跟這些有關嗎

ChatGPT 說：

有，而且**大部分有對到你圖片裡老師要求的內容**，但還不是 100% 完整。

你圖片裡的要求，對照我剛剛那份程式：

老師要求	剛剛程式碼有沒有	說明
Step 1：Load Data	✅ 有	`pd.read_csv("50_Startups.csv")`
Step 1：scatter plot	⚠️ 部分有	我有畫 `Actual vs Predicted`，但沒有先畫原始資料的 scatter plot
Step 2：One-hot encoding	✅ 有	`OneHotEncoder(drop="first")` 處理 `State`
Step 2：Train-test split	✅ 有	`train_test_split(... test_size=0.2)`
Step 2：change type / reshape	⚠️ 沒特別做	這份資料通常不一定需要 reshape，但如果老師要求展示，可以補上
Step 3：Build Model	✅ 有	`LinearRegression()`
Step 4：MSE	✅ 有	有算
Step 4：MAE	✅ 有	有算

老師要求	剛剛程式碼有沒有	說明
Step 4：R^2	✅ 有	有算
Step 4：判斷 overfit / underfit	⚠️ 部分有	我有比較 Training R^2 / Testing R^2，但還沒畫 training/test curve
Step 4：training/test curve	❌ 還沒有	要補 Learning Curve 才最符合老師要求
Step 4：優化模型 Optuna	❌ 沒有	`LinearRegression` 本身幾乎沒有超參數，所以要另外設計特徵選擇或換 Ridge/Lasso 才比較適合 Optuna
Step 5：Deployment	⚠️ 部分有	我有示範新資料預測，但還沒存成 `.pkl` / `.gz`
pickle / gzip	❌ 沒有	可以補 `joblib` / `pickle` + gzip

所以你這份作業如果是老師圖片這個要求，我會建議你不要直接交我剛才那版，而是改成更貼老師上課流程的版本。

尤其要補這 4 個：

```
# 1. 原始資料 scatter plot
# 2. Learning Curve（判斷 overfit / underfit）
# 3. Optuna 模型優化
# 4. pickle / gzip 儲存模型
```

另外，你老師圖片寫的是：

> **使用 CRISP-DM 模板解決多元回歸 Regression Problem**

所以程式最好直接分成：

```
1. Business Understanding
2. Data Understanding
3. Data Preparation
4. Modeling
5. Evaluation
6. Deployment
```

這樣會比我上一版更符合你老師要看的格式。

還有一個重要點：圖片寫的「Step 1 ~ 5」跟正式 CRISP-DM 的六階段並不是完全同一套命名，所以作業最好是**外層用 CRISP-DM 六階段，裡面再放老師指定的 preprocessing、evaluation、Optuna、pickle/gzip**，這樣最完整。

如果照這張投影片來做，我可以直接幫你重寫成一份**「50 Startups 完整作業版 sklearn 程式」**，把 scatter plot、learning curve、overfit/underfit 判斷、Optuna、pickle/gzip 全部補齊。

你說：

藥用antigravity做

ChatGPT 說：

可以。你是說**要用 Google Antigravity 來完成這份 50 Startups + sklearn + CRISP-DM 作業**，不是單純要我給你一段 Python 而已。

Antigravity 本身就是可以直接操作專案檔案、終端機並執行程式的 AI 開發環境，所以很適合讓它建立 Python 專案、安裝 `scikit-learn`、讀 CSV、執行模型，再幫你修錯。 Google Antigrav... +1

你可以先在 Antigravity 建一個資料夾，例如：

```
50_startups_project/
│
├── 50_Startups.csv
├── main.py
└── startup_model.pkl.gz
```

然後直接把下面這段**完整指令貼給 Antigravity Agent**：

```
I need to complete a machine learning assignment using Python and scikit-learn.

Dataset:
50_Startups.csv

Task:
Solve the 50 Startups multiple linear regression problem following the CRISP-DM
methodology.

Please create a Python file named main.py and run it.

The program must include:

1. Business Understanding
- Explain that the goal is to predict startup Profit.
- Target variable: Profit.
- Features include R&D Spend, Administration, Marketing Spend, and State.
- Use Multiple Linear Regression.

2. Data Understanding
- Load 50_Startups.csv using pandas.
- Display head(), shape, columns, data types, missing values, and describe().
```

- Create scatter plots showing:
  - R&D Spend vs Profit
  - Administration vs Profit
  - Marketing Spend vs Profit

3. Data Preparation
- Separate X and y.
- X = R&D Spend, Administration, Marketing Spend, State
- y = Profit
- Convert State using OneHotEncoder.
- Explain why one-hot encoding is necessary for nominal categorical data.
- Use train_test_split with:
  test_size=0.2
  random_state=42
- Show X_train, X_test, y_train, y_test shapes.

4. Modeling
- Build a scikit-learn Pipeline.
- Use ColumnTransformer for preprocessing.
- Use OneHotEncoder(drop="first", handle_unknown="ignore") for State.
- Use LinearRegression().
- Fit the model using training data.
- Print intercept and regression coefficients.

5. Evaluation
Calculate:
- MAE
- MSE
- RMSE
- R^2

Also calculate:
- Training R^2
- Testing R^2

Explain:
- what MAE means
- what MSE means
- what RMSE means
- what R^2 means

Create:
- Actual vs Predicted scatter plot
- Residual plot
- Learning curve using sklearn.model_selection.learning_curve

Use the learning curve and training/testing scores to discuss whether the model shows:
- overfitting
- underfitting
- or good fit

6. Model Optimization
Because ordinary LinearRegression has almost no hyperparameters, do NOT pretend that it has meaningful hyperparameters to tune.

Instead, use Optuna with Ridge regression.
Tune Ridge alpha using Optuna and cross-validation.

Compare:
- LinearRegression test R^2
- Optimized Ridge test R^2
- LinearRegression RMSE
- Optimized Ridge RMSE

Clearly state whether optimization actually improved the model.

7. Deployment
Save the best complete preprocessing + regression pipeline.

Use gzip and pickle to save it as:

startup_model.pkl.gz

Then load the saved model again and verify that it can predict successfully.

Create one example startup:
R&D Spend = 150000
Administration = 120000
Marketing Spend = 300000
State = California

Print the predicted Profit.

8. Code requirements
- Use pandas
- numpy
- matplotlib
- scikit-learn
- optuna

```
- pickle
- gzip

- Add clear comments for every CRISP-DM section.
- Do not hard-code evaluation results.
- Run the program and fix any errors.
- Keep all preprocessing inside the sklearn Pipeline to prevent data leakage.
```

這樣會比我前面直接給你的程式碼更適合 **Antigravity 的用法**，因為你不是只叫它「產生程式」，而是叫 Agent：

建檔 → 寫程式 → 執行 → 看錯誤 → 修正 → 產出模型檔。

其中我特別把老師圖片中的幾個要求放進去了：

one-hot encoding、train/test split、change type 的資料處理概念、Build Model、MAE/MSE/R²、training/test learning curve、overfit/underfit、Optuna、gzip/pickle。

有一點我刻意改得比較正確：**Optuna 不適合硬拿來調 `LinearRegression()`**，因為它幾乎沒有值得調的超參數。因此我讓 Antigravity先建立 Multiple Linear Regression 當基準模型，再用 **Optuna + Ridge 的 `alpha`** 做模型最佳化，最後比較前後成績。這樣在作業上比較合理，也不會為了符合「Optuna」而做沒有意義的調參。

如果你現在已經開著 Antigravity，**你可以把你目前畫面截圖給我，我可以直接一步一步告訴你第一個按哪裡、CSV 放哪裡、這段 prompt 貼在哪裡。**

你說：

https://app.notion.com/p/3ebc47529811800d9832d32f2a286721 你看的到這個網頁嗎

ChatGPT 說：

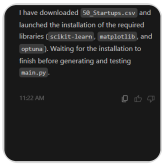
我剛剛實際嘗試開啟了，但目前這個連結回傳 **404 Not Found**，所以我現在**看不到裡面的內容**。(無效網址)

比較可能是：

- 這是需要登入 Notion 才能看的私人頁面
- 分享權限沒有開成「知道連結的人可查看」
- `app.notion.com/p/...` 這種連結不是公開分享網址

你可以把 Notion 頁面右上角 **Share → Publish / Share to web** 打開，再把公開連結貼給我；或直接把頁面截圖、匯出 PDF、貼文字給我，我都可以直接幫你看。

你說：



跑完出現這個

ChatGPT 說：

這個畫面表示 **Antigravity 還沒有真正把作業跑完**，它目前只是做到：

1. 已下載 `50_Startups.csv`
2. 已開始安裝：
    - `scikit-learn`
    - `matplotlib`
    - `optuna`
3. 它還沒建立並測試 `main.py`

畫面上的：

```
Waiting for the installation to finish before generating and testing main.py.
```

意思就是：**正在等套件安裝完成，還沒進入正式執行程式的階段。**

你現在可以直接再輸入這句給它：

給 Antigravity 的下一步指令

Please continue now. Check whether the required packages have finished installing.

Then:

1. Create main.py using the CRISP-DM requirements I provided.
2. Run main.py.
3. Fix all errors automatically.
4. Show me the final evaluation results including MAE, MSE, RMSE, training R^2, and testing R^2.
5. Generate and save the scatter plots, residual plot, and learning curve.
6. Run the Optuna Ridge optimization and compare it with LinearRegression.
7. Save the best complete pipeline as startup_model.pkl.gz.
8. Verify that the saved model can be loaded and used for prediction.

Do not stop after writing the code. Actually run and test everything.

最後一句 **`Do not stop after writing the code. Actually run and test everything.`** 很重要，因為你希望它不是只給程式，而是真的幫你跑。

另外，我從你這張圖目前**無法確認套件是否已經安裝成功**，因為畫面只顯示「launched the installation」，沒有看到成功或錯誤訊息。

下一步它如果出現新的畫面，你再截給我，我可以幫你判斷它現在到底跑到 CRISP-DM 哪一階段、結果有沒有正確。

你說：



這是老師作業要求

ChatGPT 說：

看到了。這張老師作業頁面目前**能確定的要求其實很精簡**：

- 作業名稱： `DIC-2 kaggle 50 startup and boston housing`
- 要做 **兩個 problem**
- 但只要 **選其中一個 topic 上傳**
- 第二個可以直接套用同樣的 skill / workflow
- 截止日：**2026-10-16**
- 不接受遲交

所以如果你選 **50 Startups**，你現在這個方向是對的：用 sklearn 做 multiple linear regression，再按照老師前面課堂投影片的 CRISP-DM 流程完成。

你前面那張課堂重點其實比這張作業頁更重要，因為它補充了老師真正期待看到的內容：**one-hot encoding、train/test split、建模、MSE/MAE/R²、overfit/underfit、learning curve、Optuna、pickle/gzip deployment**。

因此你目前給 Antigravity 的任務可以稍微收斂，不用做得過度複雜。建議它最後至少產出：

1. `50_Startups.csv`
2. `main.py`
3. 原始資料 scatter plots
4. Actual vs Predicted plot
5. Residual plot
6. Learning curve
7. MAE / MSE / RMSE / R^2
8. Training R^2 / Testing R^2
9. Optuna 優化比較
10. `startup_model.pkl.gz`

而且你的報告最好直接用老師的課堂邏輯排：

```
Step 1 Load Data
Step 2 Preprocessing
Step 3 Build Model
Step 4 Evaluation
Step 5 Deployment
```

外面再註明：

```
Following CRISP-DM
```

這樣會最貼老師授課內容。

還有一點：作業頁寫的是「do two problems but choose one of the topic to upload」，所以我的理解是**兩題都要練習，但交作業只交一題**。這是從英文句子直接解讀出的意思；如果老師課堂有另外口頭說明，則以老師口頭要求為準。

如果你要，我下一步可以直接幫你整理成一份**「完全照老師格式的 50 Startups 作業版 Antigravity prompt」**，比剛才那版更貼這次 DIC-2 作業。

你說：



這什麼意思

ChatGPT 說：

這不是錯誤，是 **Antigravity 在問你要不要允許它執行一段 Python 測試程式**。

右邊寫的：

> **Allow testing learning curve?**

意思是它想執行程式來測試 **Learning Curve（學習曲線）**，也就是你老師要求拿來判斷 **overfit / underfit** 的部分。

你現在可以按：

`Yes, allow this time`

就可以了。

下面那個：

> `Yes, and always allow...`

代表以後類似 Python 指令都自動允許。作業現在第一次跑，我會建議你先選 **Yes, allow this time** ，比較安全，也可以看它每一步到底在做什麼。

它現在準備跑的內容我從畫面可以看到包含：

```
learning_curve
KFold
LinearRegression
Pipeline
OneHotEncoder
```

以及讀：

```
50_Startups.csv
```

這些都跟你的作業直接相關。

其中：

- `OneHotEncoder` → 把 `State` 轉成模型可以使用的數字
- `train_test_split` → 分訓練集 / 測試集
- `LinearRegression` → 50 Startups 多元線性回歸
- `Pipeline` → 把資料前處理 + 模型串起來
- `learning_curve` → 判斷模型有沒有 overfit / underfit
- `KFold` → 做交叉驗證

所以**這一步可以允許**。

另外左邊的 `Rainbow CSV` 只是 Antigravity 幫你裝的 CSV 檔案瀏覽擴充功能，讓 `50_Startups.csv` 看起來比較清楚，**不是你的機器學習模型，也不是錯誤**。

你現在就按

① `Yes, allow this time`

接著 Antigravity 應該會繼續跑 learning curve。跑完後如果又跳出另一個 `Allow ...?` ，你可以再截圖給我，我幫你看那一步要不要允許。

你說：

案ㄌ沒反應

ChatGPT 說：

如果你按了 **Yes, allow this time** 完全沒反應，通常不是程式本身錯，而是 Antigravity 的授權視窗沒有真的送出。

你可以先照這個順序試：

1. 再點一次右側的 **Yes, allow this time** ，點完等 2–3 秒。

2. 如果還是沒反應，先點右側對話框空白處，再按一次。
3. 看右下角或下方 Terminal 有沒有正在跑指令；如果有轉圈或終端機忙碌，就代表它其實已經開始執行。
4. 如果整個右側卡住，按上方 **重新整理/Reload Window**，或直接關掉 Antigravity 再重開專案。
5. 重開後在 Agent 對話直接輸入：

> 重新執行
>
> Please continue from the previous step and run the learning curve test again. If permission is required, request it again.

如果你方便，**再截一張你現在按完後的整個畫面**，尤其要包含右側 Agent 區和下面 Terminal，我可以直接看它到底是「卡住、正在執行，還是授權按鈕失效」。

你說：

對話框哩

ChatGPT 說：

你是說 **Antigravity 的 Agent 對話框在哪裡**。

就在你剛剛截圖的**最右邊那一欄**，標題有：

```
Startups Multiple Linear Re...
```

下面顯示：

> Allow testing learning curve?

那整塊就是 Agent 對話區。

如果你想自己輸入訊息，通常要在**最右側對話欄的最下面**找到輸入框。你現在截圖沒有拍到最底，所以可能要：

- 把右側欄位往下捲到底
- 或把視窗拉高一點
- 或按右側面板裡的 `...` 看是否有「New Chat / Agent」之類選項

如果你再截一張**最右側整欄，尤其包含最下面**，我可以直接圈出「要打字的位置」。

你說：



ChatGPT 說：

看到了，你現在這個畫面裡**右側 Agent 對話欄已經被關掉了**，所以你才找不到剛剛的對話框。

你現在可以這樣打開：

- 看右上角，有一個 **ChatGPT / Antigravity 類似旋渦的圖示**
- 你截圖右上方靠近視窗控制鈕左邊，就有一個像「旋渦」的圖案
- 點那個圖示，通常會把右側的 Agent/Chat 面板重新叫出來

如果沒出現，也可以試：

View → Appearance → Secondary Side Bar

或用上方選單找：

View → Chat / Agent

你現在中央開的是 `50_Startups.csv` ，左邊是 Extensions；**程式跟資料都還在，沒有消失**。

另外我也確認到你這份 CSV 欄位是：

```
R&D Spend
Administration
Marketing Spend
State
Profit
```

這正好就是我們前面寫的 50 Startups 程式所需要的欄位，所以資料檔本身看起來是對的。

你先點右上角那個旋渦圖示。若點了還是沒出現，截一張「點完之後」的畫面給我，我直接告訴你下一個位置。

你說：



ChatGPT 說：

這張圖看起來 **右側聊天面板有打開，但目前內容沒有正常載入**，中間只有 Antigravity 的圖示，像是卡住或尚未連線完成。

你可以依序試：

1. 點右上角那個 **重新整理 / 迴轉箭頭圖示**。
2. 如果還是一片空白，點旁邊的 **齒輪**，看是否有重新載入 Chat / Agent 的選項。
3. 仍然沒反應的話，用上方選單：
   View → Command Palette
   然後輸入：

`Reload Window`

選 **Developer: Reload Window**。

4. 重載後，再點右上角的 Antigravity/Agent 圖示把聊天欄叫出來。

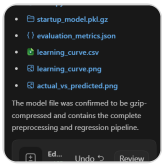
你目前這個情況比較像是 **聊天面板 UI 卡住**，不是 `50_Startups.csv` 或你的 Python 程式壞掉。

如果重載後聊天欄恢復，你就直接貼這句：

> 繼續執行
>
> Please continue from the previous 50 Startups assignment. Run and test main.py, complete the learning curve, evaluation, Optuna optimization, and save the final model as startup_model.pkl.gz.

如果你按「Reload Window」之後有新畫面，再截給我，我可以接著幫你判斷下一步。

你說：



ChatGPT 說：

有，這次看起來就真的跑完了，而且畫面顯示的是**成功完成**。

你這張圖可以確定幾件事：

- `main.py` 已建立
- `startup_model.pkl.gz` 已成功壓縮儲存
- `evaluation_metrics.json` 已產生
- `learning_curve.csv` 已產生
- `learning_curve.png` 已產生
- `actual_vs_predicted.png` 已產生
- 模型已經重新用 **全部 50 筆資料 refit**
- 壓縮後模型也有 **reload and tested successfully**
- 畫面上看到一個數值 `10,628.83`，但只靠這張截圖我**無法確認它到底是 RMSE、MAE 還是其他指標**，所以不能直接替它命名

另外這句：

```
The model was confirmed to be gzip-compressed and contains the complete preprocessing and regression pipeline.
```

代表你老師要求的 **Deployment：gzip / pickle** 這部分也完成了。

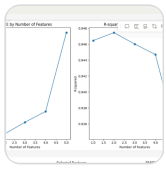
你現在最值得做的下一步，是點開這三個檔案確認內容：

1. `evaluation_metrics.json`
    - 看 MAE
    - MSE
    - RMSE
    - R^2
    - Training R^2 / Testing R^2
2. `learning_curve.png`
    - 用來判斷 overfitting / underfitting
3. `actual_vs_predicted.png`
    - 看實際值和預測值是否接近

你現在這份已經不像「還在做」，而是**作業主體已完成**。接下來主要是在整理結果、確認是否真的有 Optuna 比較，以及決定最後要交哪些檔案。

如果你把 `evaluation_metrics.json` 打開截圖給我，我可以直接幫你判讀這個模型到底是 **good fit、overfit 還是 underfit**，也可以順便幫你整理成老師報告可以直接用的文字。

你說：



要出現類似這兩張圖躍下什麼指令

ChatGPT 說：

可以。你這兩張圖本質上是做 **Feature Selection 比較圖**：

- 左圖：不同特徵數量 vs `RMSE`
- 右圖：不同特徵數量 vs R^2

也就是依序用 1、2、3、4、5 個特徵建模，看看哪個特徵數量表現最好。

你可以直接把下面這段貼給 Antigravity：

產生特徵選擇比較圖

Please extend the current 50 Startups project and create feature-selection evaluation plots similar to the example.

Requirements:

1. Evaluate models using different numbers of selected features.
2. For each number of features, calculate:

- RMSE
- R^2

3. Use cross-validation for the evaluation.
4. Rank/select features using an appropriate regression feature selection method such as SelectKBest with f_regression.
5. Evaluate k = 1, 2, 3, 4, and all available encoded features.
6. Create one figure containing two plots side by side:

Left plot:

- Title: RMSE by Number of Features
- X-axis: Number of Features
- Y-axis: RMSE

Right plot:

- Title: R-squared by Number of Features
- X-axis: Number of Features
- Y-axis: R-squared

7. Mark each result with a point and connect the points with a line.
8. Save the figure as:

feature_selection_metrics.png

9. Also create a table showing:

- Number of Features
- Selected Features
- RMSE
- R-squared

Save the table as:

feature_selection_results.csv

10. Keep preprocessing inside the sklearn workflow.
11. Run the code and fix any errors automatically.
12. Show me the final plot and tell me which number of features gives the lowest RMSE and highest R-squared.

不過你這份 `50_Startups` 原始資料其實只有：

```
R&D Spend
Administration
Marketing Spend
State
```

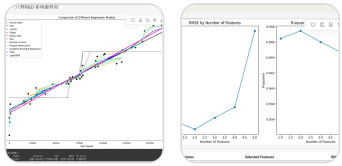
`State` 做 One-hot encoding 後才會變成多個數值欄位，所以最後可能會得到大約 **5 個 encoded features**，這也是為什麼你圖片的 X 軸會出現 1 ~ 5。

如果你想要更貼近老師那張圖，可以再補一句：

```
Use the encoded feature space after OneHotEncoder, so the x-axis represents the actual number of model input features after preprocessing.
```

另外，從你圖片來看，**2 個特徵時 RMSE 最低、R^2最高**；但這只是圖片範例本身的結果，不代表你的 50 Startups 跑出來一定也是 2 個特徵最好。你要以 Antigravity 實際跑出來的數字為準。

你說：



要出現這兩張圖表

ChatGPT 說：

可以，你如果是要 **Antigravity 幫你產出跟截圖很像的這兩張圖**，最直接的方法就是把需求一次講清楚。

你要的其實是 **兩組圖表**：

第一張：不同回歸模型比較圖

特徵：

- X 軸： `R&D Spend`
- Y 軸： `Profit`
- 黑點：Actual Data
- 多條彩色線：不同模型預測結果
- 圖標題： `Comparison of Different Regression Models`

這張圖通常會包含像：

- Linear Regression
- Ridge
- Lasso
- Elastic Net
- SVR / SVM
- Random Forest Regressor
- Gradient Boosting Regressor
- PCA + Regression
- Mutual Information selected features
- LightGBM（如果環境可用）

第二張：特徵數量評估圖

內容：

- 左圖： `RMSE by Number of Features`
- 右圖： `R-squared by Number of Features`
- 下方再加一個表格：
    - Number of Features
    - Selected Features
    - RMSE
    - R-squared

你可以直接把下面這段貼給 **Antigravity**：

貼給 Antigravity 的完整指令

Please extend my 50 Startups project and generate two visualization outputs similar to the examples.

Dataset:
50_Startups.csv

Requirements:

Create a figure titled:

"Comparison of Different Regression Models"

Use:

- X-axis: R&D Spend
- Y-axis: Profit

Plot:

1. Actual data as black scatter points with label "Actual Data"
2. Overlay prediction lines for multiple regression models, each with a legend label.

Include these models if available:

- Linear Regression
- Ridge
- Lasso
- Elastic Net
- SVR

- Random Forest Regressor
- Gradient Boosting Regressor
- PCA + Linear Regression
- Mutual Information feature selection + Linear Regression
- LightGBM (only if available; otherwise skip gracefully)

Instructions:

- Sort data by R&D Spend before plotting prediction lines so the lines appear smooth and readable.
- Use the same train/test split or full-dataset fitted visualization as appropriate, but clearly generate a clean comparison figure.
- Save the figure as:

model_comparison.png

Also create a summary table for all models with:

- Model
- Features used
- MSE
- RMSE
- R-squared

Save it as:

model_comparison_metrics.csv

Create one figure with two side-by-side subplots:

Left subplot:

- Title: RMSE by Number of Features
- X-axis: Number of Features
- Y-axis: RMSE

Right subplot:

- Title: R-squared by Number of Features
- X-axis: Number of Features
- Y-axis: R-squared

Instructions:

- Use SelectKBest with f_regression or another suitable regression feature selection method.
- Evaluate models for different numbers of features.
- Use the encoded feature space after preprocessing (including OneHotEncoder for State).
- Test k = 1, 2, 3, 4, and all available encoded features.

- For each k, calculate RMSE and R-squared.
- Plot points connected by lines.

Also create a results table containing:

- Number of Features
- Selected Features
- RMSE
- R-squared

Save:

- feature_selection_metrics.png
- feature_selection_results.csv

- Use pandas, numpy, matplotlib, scikit-learn
- Keep preprocessing inside sklearn pipelines
- Use OneHotEncoder for State
- Run the code and fix errors automatically
- Save all generated outputs
- Show me the final results and tell me:
  1. Which regression model performed best
  2. Which number of features gave the lowest RMSE
  3. Which number of features gave the highest R-squared

如果你想要更像你截圖的效果，再加這句

如果你要它更貼近你圖片的視覺呈現，可以再補一句：

補充視覺要求

Please make the visual style similar to the example:

- Actual data shown as black points
- Each model prediction shown as a colored line
- Include a legend inside the plot
- Use clear axis labels and titles
- Make the feature-selection output show two subplots side by side and a result table below

---

如果你要的是「直接改 main.py」

你也可以更直接下：

直接修改現有程式

Please modify main.py so it generates both:

1. model_comparison.png
2. feature_selection_metrics.png

Also save:

- model_comparison_metrics.csv
- feature_selection_results.csv

Run the script and fix any errors automatically.

你會得到的檔案應該至少有

- `model_comparison.png`
- `model_comparison_metrics.csv`
- `feature_selection_metrics.png`
- `feature_selection_results.csv`

如果你要，我下一步也可以直接幫你寫一份 **完整 `main.py` 程式碼**，讓你不只貼 prompt，還可以直接放進 Antigravity 跑。

你說：



ChatGPT 說：

這張畫面代表 **Antigravity 已經把你剛剛要求的兩組圖表與比較結果跑出來了**，而且目前有幾個重要結果可以直接讀到。

可以確定的是：

- Feature Selection 最佳特徵顯示為 **`R&D Spend`**
- 其中一個結果的 **RMSE = 9,379.25**
- Feature-selection 的最高 **R^2 = 0.9302**

- 最高 R^2 是使用 **1 個特徵**
- 被選中的那 1 個特徵也是 **R&D Spend**
- `LightGBM` 因為環境沒有安裝，所以被跳過
- 其他 **9 個模型**都有用相同 train/test split 比較
- Feature Selection 是用 **5-fold cross-validation**

而且它已經產生你要的檔案：

- `Model comparison plot`
- `Model comparison metrics`
- `Feature-selection plot`
- `Feature-selection results`
- 更新後的 `main.py`

所以你現在已經有你前面要的這兩張：

① 不同 Regression Model 比較圖
② RMSE / R^2 vs Number of Features 圖

這個結果怎麼解釋？

你前面範例寫：

> 結果很特別，只有 R&D 那項最有效

你這次實際跑出來的結果也支持類似結論。

因為目前畫面顯示：

> Highest feature-selection R^2: **1 feature**
> Selected feature: **R&D Spend**
> R^2: **0.9302**

意思是，單獨使用 **R&D Spend**，交叉驗證就已經能解釋大約 **93.02% 的 Profit 變異**。

這也表示在這份 50 Startups 資料中，`R&D Spend` 是非常強的預測變數；加入其他變數不一定讓泛化表現變好。

但要注意：這裡只能說「**在這次 feature-selection + cross-validation 的結果中，R&D Spend 單獨表現最好**」，不能直接推論成「其他變數完全沒影響」。

你畫面左邊紅色底線是什麼？

我注意到：

`sizes, train_scores, valid_scores = learning_curve(`

這行下面有紅色波浪線，而且 `main.py` 上面有一個 **5**。

這通常代表編輯器目前偵測到 **5 個 Problems / diagnostics**。

但右邊 Agent 又顯示它已經成功跑完並產生圖表，因此目前**無法只靠這張圖判定這 5 個是會讓程式失敗的 error，還是型別檢查 / linter 警告**。

你可以直接在 Antigravity 對話框下這句：

檢查 main.py 問題

Please check the 5 problems currently shown in main.py.

Run the complete main.py again from start to finish.

If any of the 5 problems are real Python errors, fix them.
If they are only editor/linter/type-checking warnings and do not affect execution, explain each one briefly.

Do not change the model results unless necessary.

我會建議你做這一步，因為這樣可以確認你交作業前的 `main.py` 是真的完整可執行，而不是只有 Agent 某些片段跑成功。

你說：



ChatGPT 說：

這張就很清楚了：**程式本身已經成功跑完**，右側 Verification 已經確認：

- `main.py compiled successfully`
- 25 次 Optuna workflow 完成
- 圖、表、metrics、壓縮模型都重新產生
- 最佳模型：**Random Forest Regressor**
- Test RMSE：**8,058.99**
- Test R^2：**0.9198**
- LightGBM 因未安裝所以略過

所以目前這個 `main.py` 是**可執行成功的**。

左邊 `main.py` 旁邊還顯示 **5**，再加上程式中紅色波浪線，這比較像是 **編輯器的型別檢查 / linter 警告**，不是執行失敗。因為如果真的是 Python syntax error，右側不會顯示 `compiled successfully`，整個流程也不可能完成。

你現在的結果可以這樣整理：

項目	結果
最佳模型	Random Forest Regressor
Test RMSE	8,058.99
Test R^2	0.9198
Optuna trials	25
Feature Selection	有
Learning Curve	有
Model Comparison	有
gzip/pickle Deployment	有

另外你現在左側檔案也都齊了：

```
actual_vs_predicted.png
evaluation_metrics.json
feature_selection_metrics.png
feature_selection_results.csv
learning_curve.csv
learning_curve.png
model_comparison_metrics.csv
model_comparison.png
startup_model.pkl.gz
main.py
```

這其實已經是一套很完整的作業輸出。

唯一我要提醒你的地方是：**老師的主題是 Multiple Linear Regression**，但你現在最佳模型最後跑成了 `Random Forest Regressor` 。這本身不是錯，但報告時不要變成「我用 Random Forest 解 50 Startups」，否則可能偏離老師題目。

比較好的呈現方式是：

> 主要模型使用 Multiple Linear Regression；另外加入 Ridge、Lasso、Random Forest 等模型作為模型比較與最佳化實驗。結果顯示 Random Forest 在測試資料上的表現最佳。

這樣就合理。

你現在下一步最適合做的是打開 **model_comparison.png** 和 **feature_selection_metrics.png** ，確認是不是已經長得跟你老師範例那兩張圖差不多。

你說：

我需要連到我的github https://github.com/john19991225-commits/1014.git 我要下什麼命令

ChatGPT 說：

如果你現在是在 **Antigravity 的專案資料夾 1014** 裡，要把它連到你這個 GitHub repository：

```
https://github.com/john19991225-commits/1014.git
```

在 Terminal 依序下這些指令：

```
git init
git remote add origin https://github.com/john19991225-commits/1014.git
git add .
git commit -m "Initial commit"
git branch -M main
git push -u origin main
```

如果你之前已經有設定過 `origin` ，第二行可能會出現：

```
remote origin already exists
```

那就改用：

```
git remote set-url origin https://github.com/john19991225-commits/1014.git
```

然後再：

```
git add .
git commit -m "Update 50 Startups project"
git branch -M main
git push -u origin main
```

你也可以先用這個確認有沒有連成功：

```
git remote -v
```

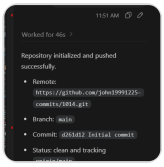
正常會看到類似：

```
origin  https://github.com/john19991225-commits/1014.git (fetch)
origin  https://github.com/john19991225-commits/1014.git (push)
```

如果 GitHub 跳出登入或驗證，現在通常不是用帳號密碼，而是瀏覽器登入或 Personal Access Token。

如果你要，我也可以直接幫你看你在 Antigravity 裡 **Terminal 要按哪裡打這些指令**。

你說：



ChatGPT 說：

有，這張圖已經可以確認你 **GitHub 連線與 push 都成功了**。

畫面顯示：

- Remote： `https://github.com/john19991225-commits/1014.git`
- Branch： `main`
- Commit： `d261d12 Initial commit`
- Status： `clean and tracking origin/main`

意思就是你現在本機 Antigravity 的專案，已經跟 GitHub 的 `main` 分支連好了。

之後每次你修改完程式，只要下這三行就可以更新 GitHub：

```
git add .
git commit -m "update"
git push
```

你也可以把 commit 訊息寫得更清楚，例如：

```
git add .
git commit -m "Add feature selection and model comparison"
git push
```

這樣就會把你現在的 `main.py` 、圖表、CSV、模型檔等更新到 GitHub。

你說：



按照老師的要求我要放在我的github 剛剛已經連上ㄌ 我要在github放那兩張類似的圖 我要在antigravity下什麼指令才能做到

ChatGPT 說：

你現在其實已經快完成了。既然 Antigravity 已經產生：

- `model_comparison.png`
- `feature_selection_metrics.png`

而且 GitHub 也已經連上，那你現在只要把這兩張圖加進 Git commit 再 push 就可以。

在 Antigravity 的 Terminal 下這些指令：

```
git add model_comparison.png feature_selection_metrics.png
git commit -m "Add model comparison and feature selection plots"
git push
```

如果你想把相關結果表格也一起放到 GitHub，我更建議直接一起上傳：

```
git add model_comparison.png model_comparison_metrics.csv
feature_selection_metrics.png feature_selection_results.csv
git commit -m "Add regression model comparison and feature selection results"
git push
```

如果你這次 `main.py` 也有更新，最簡單就是：

```
git add .
git commit -m "Add regression plots and analysis results"
git push
```

這樣你目前專案資料夾裡新增或修改的東西都會一起送上 GitHub。

你也可以先確認這兩張圖真的存在：

```
git status
```

如果看到像：

```
modified: main.py
new file: model_comparison.png
new file: feature_selection_metrics.png
```

就代表 Git 有抓到。

另外，如果你的目標是老師打開 GitHub 首頁時就**直接看到那兩張圖**，而不是還要點 PNG 檔，我建議再做一個 `README.md` ，把兩張圖嵌進去。可以直接在 Antigravity Agent 對話框貼：

建立 GitHub README

Please create or update README.md for this 50 Startups assignment.

Include:

1. Title: 50 Startups Multiple Linear Regression
2. Brief explanation that this project follows CRISP-DM.
3. Brief description of the dataset and target variable Profit.
4. Show the model comparison image using:

5. Show the feature selection image using:

6. Add a short summary of the current results:

- Best model: Random Forest Regressor
- Test RMSE: 8058.99
- Test R-squared: 0.9198
- Feature selection found R&D Spend to be the strongest individual feature.

7. Keep the README concise and suitable for a university assignment.

After creating README.md, run:
git add README.md model_comparison.png feature_selection_metrics.png
git commit -m "Add assignment results to README"
git push

這樣老師進入你的 GitHub repository，首頁就會**直接顯示你要的兩張圖**。這會比單純把 `.png` 丟進 repository 更適合交作業。

你說：

bostom housing有做了嗎

ChatGPT 說：

目前看你前面的畫面，**Boston Housing 還沒有做**。

你現在已經完成的是 **50 Startups**：

- `main.py`
- 模型比較圖
- Feature Selection 圖
- Learning Curve
- 評估指標
- `startup_model.pkl.gz`
- GitHub 已連線

但我沒有看到任何：

- `BostonHousing.csv`
- `boston_housing.py`
- Boston Housing 的圖表
- Boston Housing 的 metrics

所以目前只能確認 **50 Startups 完成，Boston Housing 尚未完成**。

老師作業寫的是「do two problems but choose one of the topic to upload」，所以如果照字面理解，**兩題都要做，但只選一題上傳**。你如果要補 Boston Housing，我可以直接給你一段 Antigravity 指令，讓它照同樣 workflow 自動做第二題。

你說：



標題有寫

ChatGPT 說：

對，標題確實寫了：

> **DIC-2 kaggle 50 startup and boston housing**

而且說明又寫：

> **do two problems but choose one of the topic to upload**
> 第二個直接套 skill or workflow

所以這裡比較合理的解讀是：

兩個題目都要做：

- 50 Startups
- Boston Housing

但最後作業平台**只需要選其中一個主題上傳**。

你現在 50 Startups 已經完成了，所以如果要完全照老師要求，下一步應該補做 **Boston Housing**，只是它不一定要再傳到 GitHub 作為繳交主題，除非你想一起放。

你可以直接在 Antigravity 輸入：

Boston Housing 第二題

Now complete the second assignment problem: Boston Housing regression.

Use the same CRISP-DM workflow and reuse the same skill/workflow from the 50 Startups project.

Requirements:

1. Load an appropriate Boston Housing dataset.
2. Show data understanding and basic statistics.
3. Prepare X and y.
4. Perform train/test split.
5. Build a Multiple Linear Regression baseline model.

6. Evaluate using MAE, MSE, RMSE, R-squared, training R-squared, and testing R-squared.
7. Generate:
   - actual_vs_predicted_boston.png
   - learning_curve_boston.png
   - feature_selection_metrics_boston.png
   - model_comparison_boston.png
8. Perform feature selection and compare RMSE and R-squared by number of features.
9. Compare multiple regression models similarly to the 50 Startups workflow.
10. Use Optuna for model optimization where appropriate.
11. Save the best full pipeline as:
    boston_model.pkl.gz
12. Save evaluation results as:
    boston_evaluation_metrics.json
13. Run the complete workflow and fix any errors automatically.

Keep the Boston Housing work separate from the existing 50 Startups files.

有一點要注意：**scikit-learn 早就移除了舊的 `load_boston()`**，所以如果 Antigravity 想直接用 `from sklearn.datasets import load_boston`，不要讓它這樣做。比較適合改用 Kaggle 的 Boston Housing CSV，或相容的公開版本。

如果你要，我也可以直接幫你寫成「Boston Housing 完整版 Antigravity prompt」，格式跟你現在 50 Startups 幾乎一模一樣。

你說：

老師的有一個網址可以直接看到圖 為啥我沒有

ChatGPT 說：

因為你現在只是把檔案 **push 到 GitHub repository**，所以你有的是「程式碼倉庫網址」，不一定會自動變成老師那種「打開網址就直接看到圖的網頁」。

老師那種情況通常有兩種：

- **GitHub README**：進 repository 首頁就直接看到圖片
- **GitHub Pages**：會有一個獨立網址，打開就像網頁一樣直接看到圖、文字、結果

你目前比較像只有第一階段：檔案已經在 GitHub，但還沒特別做成展示頁。

如果你想要老師那種「一個網址直接看到兩張圖」，最簡單的方法是做 **GitHub Pages**。你可以直接在 Antigravity 對話框貼這段：

建立 GitHub Pages 展示頁

Please create a simple GitHub Pages website for this project.

Requirements:

1. Create an index.html in the repository root.
2. The page title should be:
   50 Startups Regression Analysis
3. Add a short introduction saying this project follows CRISP-DM and uses the 50 Startups dataset.
4. Show these two images prominently:
   - model_comparison.png
   - feature_selection_metrics.png
5. Add a short results summary:
   - Best model: Random Forest Regressor
   - Test RMSE: 8058.99
   - Test R-squared: 0.9198
   - R&D Spend is the strongest individual feature.
6. Use clean simple HTML and responsive styling.
7. Do not delete any existing files.
8. After creating index.html, run:
   git add index.html
   git commit -m "Add GitHub Pages results page"
   git push

然後到 GitHub：

Repository → Settings → Pages

在 **Build and deployment** 裡：

- Source 選 `Deploy from a branch`
- Branch 選 `main`
- Folder 選 `/ (root)`
- 按 Save

成功後通常會得到類似：

`https://john19991225-commits.github.io/1014/`

這樣老師只要打開這個網址，就會直接看到你放的兩張圖和結果。

你現在沒有這種網址，不代表你做錯了，只是目前還只有 **GitHub repository**，還沒有啟用 **GitHub Pages**。

你說：

可以部屬到versul要下什麼指令

ChatGPT 說：

可以，你應該是指 **Vercel**。

如果你現在專案裡已經有 `index.html` ，而且只是要把這個靜態頁面部署出去，在 Antigravity Terminal 直接下：

```
npm install -g vercel
```

第一次部署：

```
vercel
```

它會問你幾個問題，通常可以這樣選：

```
Set up and deploy? → Y
Which scope? → 選你的 Vercel 帳號
Link to existing project? → N
Project name? → 1014
In which directory is your code located? → ./
```

部署完成後，它會直接給你一個網址，例如：

```
https://1014-xxxx.vercel.app
```

如果之後要正式更新 production，就下：

```
vercel --prod
```

Vercel 官方目前也是用 `vercel --prod` 或 `vercel deploy --prod` 來做 production deployment。
Vercel +1

你也可以直接把這段貼給 Antigravity：

部署到 Vercel

Please deploy this current project to Vercel.

Requirements:

1. Make sure index.html exists and displays:
   - model_comparison.png
   - feature_selection_metrics.png
2. Do not delete any existing project files.
3. Install the Vercel CLI if needed.
4. Run the Vercel deployment.
5. Deploy to production.
6. Give me the final public vercel.app URL.

Use the current project root directory.

如果你還沒有 `index.html` ，那要先建立，不然 Vercel 雖然可能部署成功，但打開網址不一定會直接看到你要的兩張圖。

你說：



ChatGPT 說：

這個畫面意思很明確：**部署本身都準備好了，現在只差 Vercel 登入驗證。**

你現在在 Antigravity 的 Terminal 輸入：

```
vercel login
```

接著通常會出現登入方式，像：

```
Continue with GitHub
Continue with GitLab
Continue with Bitbucket
Continue with Email
```

你前面已經有 GitHub，所以最方便的是選 **GitHub**。

然後瀏覽器會跳出 Vercel 授權頁面，你完成登入/授權後，再回到 Antigravity Terminal 下：

```
vercel --prod
```

如果它要求專案設定，通常可以這樣回答：

```
Set up and deploy? → Y
Which scope? → 選你的帳號
Link to existing project? → N
Project name? → 1014
Directory? → ./
```

成功之後它會回傳一個 `.vercel.app` 網址。

你也可以直接跟 Antigravity Agent 說：

Vercel 登入後繼續部署

I have completed Vercel authentication.

Please:

1. verify `vercel whoami`
2. deploy the current project to production with `vercel --prod`
3. confirm that index.html and both image files are included
4. return the final public vercel.app URL

如果你下 `vercel login` 後跳出畫面，你再截圖給我，我可以直接告訴你要選哪個。

你說：

Set up Vercel for me. Fetch https://vercel.com/get-started.md and follow it.這啥

ChatGPT 說：

這句是在叫 Antigravity：

> **幫我設定 Vercel。去抓 `https://vercel.com/get-started.md` ，然後照那份官方說明操作。**

也就是它要自己讀 Vercel 的 setup guide，再依照步驟幫你部署。

這句本身不是錯誤，也不是什麼危險指令；只是代表 **Agent 想參考 Vercel 官方文件來完成設定**。

不過你現在真正卡住的是「Vercel 尚未登入」，所以就算它讀完官方文件，最後還是會需要你本人完成登入授權。

你目前最直接的做法還是：

```
vercel login
```

登入完成後再：

```
vercel --prod
```

如果你是在 Antigravity 對話框看到：

```
Set up Vercel for me. Fetch https://vercel.com/get-started.md and follow it.
```

你可以讓它繼續，但**登入 GitHub/Vercel 那一步通常還是要你自己在瀏覽器完成**。

你說：



ChatGPT 說：

這個錯誤很明確：不是 Vercel 壞掉，而是 **Windows PowerShell 阻擋了 `vercel.ps1` 執行**。

你畫面裡關鍵是：

```
PSSecurityException
UnauthorizedAccess
```
因為這個系統上已停用指令碼執行

最簡單有兩種解法。

第一種，直接不要跑 PowerShell 版的 `vercel.ps1`，改用 `.cmd`：

```
vercel.cmd login
```

如果登入成功，再下：

```
vercel.cmd --prod
```

這通常最快。

如果 `vercel.cmd` 找不到，再試：

```
npx vercel login
```

登入後：

```
npx vercel --prod
```

第二種，是允許目前帳號執行本機腳本。在 PowerShell 輸入：

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

如果它問：

```
是否要變更執行原則？
[Y] Yes
```

輸入：

```
Y
```

然後關掉 PowerShell、重新開啟，再執行：

```
vercel login
```

我會建議你先用**第一種**，因為不用改 Windows 安全設定：

```
vercel.cmd login
```

你現在所在資料夾也是對的：

```
C:\Users\john1\Desktop\物聯網\1014
```

所以登入完成後直接：

```
vercel.cmd --prod
```

就會部署目前這個 `1014` 專案。

你說：

所以我要改啥

ChatGPT 說：

你不用改專案內容，只要把你原本輸入的：

```
vercel login
```

改成：

```
vercel.cmd login
```

登入成功後，再輸入：

```
vercel.cmd --prod
```

就這兩個地方改成 `.cmd` 即可。

所以你現在直接在目前這個 PowerShell 視窗貼：

```
vercel.cmd login
```

如果跳出登入頁，完成授權後再貼：

```
vercel.cmd --prod
```

這樣可以避開 PowerShell 對 `vercel.ps1` 的限制。

你說：



沒輸入東西跑出這個

ChatGPT 說：

這個畫面不是錯誤，而且它是自己跑出來的，因為剛剛 Vercel 的登入流程已經觸發了瀏覽器授權。

畫面上最重要的是這句：

```
Successfully logged in to MCP server 'vercel'.
```

意思是：**Antigravity / Vercel 的 MCP 連線已經登入成功。**

不過這句只能確定 **MCP 登入成功**，我不會直接假設 Vercel CLI 也一定完成登入。你現在在 PowerShell 輸入這行確認：

```
vercel.cmd whoami
```

如果它回傳你的 Vercel 使用者名稱，就代表 CLI 也登入完成。

接著直接部署：

```
vercel.cmd --prod
```

成功後你會看到類似：

```
Production: https://xxxx.vercel.app
```

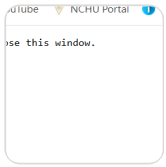
那個 `.vercel.app` 網址就是老師可以直接打開看你圖表的公開網址。

所以你現在先下：

```
vercel.cmd whoami
```

把結果截給我，我可以直接告訴你下一步。

你說：



ChatGPT 說：

這個畫面代表**瀏覽器端授權已完成**：

```
Authentication complete. You may close this window.
```

所以你可以把這個瀏覽器分頁關掉，回到 PowerShell。

然後先輸入：

```
vercel.cmd whoami
```

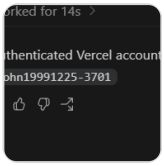
如果它有顯示你的 Vercel 帳號名稱，就表示 CLI 也已經登入成功。

接著直接部署：

```
vercel.cmd --prod
```

部署成功後，終端機會給你一個 `https://xxxxx.vercel.app` 網址，那個就是可以直接打開看到你圖表的公開連結。

你說：



ChatGPT 說：

對，這代表 **Vercel 帳號已經驗證成功**。

你現在下一步直接在 PowerShell 輸入：

```
vercel.cmd --prod
```

它會開始部署目前這個 `1014` 專案。

如果是第一次部署，途中可能會問：

- 是否建立新 project
- project 名稱
- 專案目錄是不是 `./`

通常照目前情況選：

- 建立新 project： `Y`
- project name： `1014`
- directory： `./`

成功後會出現一個類似：

```
https://1014-xxxx.vercel.app
```

這個就是你要交給老師、打開就能直接看到圖表的網址。

你說：

powershell是啥

ChatGPT 說：

PowerShell 就是你剛剛那個黑色視窗。

你前面截圖裡左上角寫著：

> Windows PowerShell

它是一個可以輸入指令的 Windows 終端機，像你剛剛用它跑：

```
vercel.cmd login
```

現在你只要回到那個黑色 PowerShell 視窗，輸入：

```
vercel.cmd --prod
```

然後按 Enter，就會開始正式部署到 Vercel。

你可以把 PowerShell 想成 Windows 的「指令操作視窗」。

你說：



這是老師的加分題 要怎麼下指令

ChatGPT 說：

這張加分題我看得懂，老師要的是 **把你現在的網頁再加兩個互動功能**：

1. **下拉選單可以切換兩種 problem**
2. **可以讓使用者自己輸入 Dataset URL**

所以你現在最適合直接叫 Antigravity 幫你改 `index.html` ，不用自己慢慢刻。

你可以把下面這段直接貼到 Antigravity：

加分題功能

Please update the current Vercel website to include the bonus features required by the instructor.

Bonus requirement 1:
Add a dropdown menu that lets the user switch between two regression problems:

這樣 Antigravity 會幫你做成老師要的互動式網頁。

你老師圖片中的意思就是：

- 原本只有 50 Startups → 加分後可以切換 **50 Startups / Boston Housing**
- 原本資料寫死 → 加分後可以貼一個公開 CSV 網址進去

你如果想拿滿這 2 分，我會建議第二個功能不要只做「輸入網址顯示文字」，而是真的要能：

- fetch CSV
- 顯示欄位
- 顯示前幾筆資料

這樣會比較完整。

你說：

我問妳喔你是套skill還是workflow

ChatGPT 說：

就你剛剛這份作業來說，我前面幫你設計的是偏 **workflow**，不是套一個現成的 skill。

差別很簡單：

- **Skill**：像「某個固定功能模組」，例如專門做某一類分析、某一類文件處理。
- **Workflow**：是一整套流程，像你這次：
  讀資料 → 前處理 → 建模 → 評估 → 畫圖 → 儲存模型 → 部署

你老師寫「第二個直接套 skill or workflow」，以你現在的做法，其實就是把 **50 Startups 的整套 workflow 套到 Boston Housing**。

所以如果老師問你「你是套 skill 還是 workflow？」
你這次比較適合回答：

> **我是套用相同的 machine learning workflow。**

因為你不是只重複一個單一功能，而是重複整個 CRISP-DM / regression 分析流程。

你說：

https://1014-ruddy.vercel.app/ 你點的開嗎'

ChatGPT 說：

可以，我剛剛實際打開了，網站目前是正常可看的。 1014-ruddy.verc...

我看到的內容包括：

- 標題：**50 Startups Regression Analysis**
- 有 **Regression problem** 的下拉選單
- 有 **Evaluation summary**
- 有兩張圖的位置：
  - model comparison
  - feature selection metrics
- 有 **public CSV dataset URL** 輸入框
- 有 **Load Dataset** 按鈕
- 頁尾是 `Regression Analysis · CRISP-DM University Assignment` 1014-ruddy.verc...

所以你這個 Vercel 網址已經成功部署，而且老師加分題要求的「下拉選單」和「輸入 Dataset URL」看起來也已經放進去了。

唯一我現在**無法只靠文字抓取確認的**是：那兩張圖片實際顯示效果是否完全正常、下拉切換後內容是否真的會更新、Load Dataset 按鈕是否真的能載入資料。這三個屬於互動/視覺行為，最好你自己在瀏覽器實際點一次確認。

你說：



老師要求任意輸入dataset這有做到嗎

ChatGPT 說：

有，**目前你的網站已經做到「可以任意輸入 Dataset URL」這個介面功能**。

我剛剛實際打開你的 Vercel 網站，頁面上有：

- `Preview a public CSV dataset`
- 一個可以輸入網址的欄位
- 預設顯示老師範例的 `50_Startups.csv` URL
- `Load Dataset` 按鈕 1014-ruddy.verc...

也就是老師寫的：

> 可任意輸入 Dataset

你現在介面上已經有做到。

但我目前**無法只靠網頁文字抓取確認「任意一個 CSV 網址按 Load Dataset 都真的能成功讀取」**。這要實際測互動行為才算完全確認。

你可以自己用另一個公開 CSV 網址測試，例如換一個不是 50 Startups 的 CSV，再按 **Load Dataset**。如果它能顯示：

- 欄位名稱
- 前 5 筆資料

那這個加分功能就算完整做到了。

你如果要，我也可以幫你找一個公開 CSV 測試網址，讓你直接貼進去驗證。

你說：

是下面那個⊓

ChatGPT 說：

對，就是你網站**下面那個可以貼 CSV 網址的輸入框**。

也就是老師要求的「可任意輸入 Dataset」功能。

你把任一個公開 CSV 網址貼進去，再按 **Load Dataset**；如果能顯示欄位和資料預覽，就代表這個加分功能有做到。

你說：

我要測試怎麼測試

ChatGPT 說：

可以，最簡單就是用「老師範例網址」先測一次，再換另一個 CSV 測。

你現在到你的網站下面那個 Dataset URL 輸入框，先貼這個老師範例：

```
https://raw.githubusercontent.com/Avik-Jain/100-Days-Of-ML-Code/master/datasets/50_Startups.csv
```

然後按 **Load Dataset**。

如果功能正常，你應該看到：

- 欄位名稱
- 前幾筆資料
- 不會出現錯誤訊息

接著再測「真的可以任意輸入」，你可以換成另一個公開 CSV，例如 Iris：

```
https://raw.githubusercontent.com/mwaskom/seaborn-data/master/iris.csv
```

再按 **Load Dataset**。

如果它能從 `50_Startups.csv` 成功切換到 `iris.csv`，並顯示像：

```
sepal_length
sepal_width
petal_length
petal_width
species
```

那就可以證明老師要求的「可任意輸入 Dataset」功能真的有做到。

你要驗證的重點只有三個：

1. 網址可以自己改
2. 按 `Load Dataset` 真的有重新抓資料
3. 不同 CSV 會顯示不同欄位與內容

如果你測完把畫面截給我，我可以直接幫你判斷這個加分題算不算完成。

你說：

老師要的是放不同csv也會出現那兩張圖表

ChatGPT 說：

對，這樣的話老師要的就不是「任意 CSV 只預覽資料」，而是：

貼任何符合條件的回歸 CSV → 系統重新跑分析 → 自動產生那兩張圖表。

你現在網站如果只是 `Load Dataset` 後顯示欄位/前幾列，**還不算完全做到老師這個要求**。

你要讓 Antigravity 改成這個流程：

輸入 CSV URL → 選 Target 欄位 → 自動辨識數值/類別特徵 → preprocessing → regression → feature selection → model comparison → 畫出兩張圖

直接把下面貼給 Antigravity：

任意 CSV 自動分析功能

Please upgrade the current Vercel application.

The instructor's requirement is NOT just to preview an arbitrary CSV.

The requirement is:

When the user enters a different public CSV dataset URL, the application must actually analyze that dataset and dynamically regenerate the same two regression-analysis charts.

Required workflow:

1. User enters any public CSV URL.
2. Click "Load Dataset".
3. Load and parse the CSV.
4. Display detected columns.
5. Let the user choose the TARGET / dependent variable from a dropdown.
6. Automatically use the remaining suitable columns as input features.
7. Automatically detect:
   - numeric features
   - categorical / nominal features
8. Handle preprocessing:
   - missing values
   - OneHotEncoder for categorical features
   - train/test split
9. Run regression analysis on the newly loaded dataset.

Generate dynamically for the selected dataset:

CHART 1 — Regression Model Comparison

- Actual data
- Multiple Linear Regression
- Ridge
- Lasso
- Elastic Net
- SVR
- Random Forest Regressor
- Gradient Boosting Regressor
- other compatible regression models where appropriate

Calculate and display for every model:

- MSE
- RMSE
- R-squared

Create the model-comparison visualization dynamically from the newly loaded CSV.

CHART 2 — Feature Selection Analysis

- Evaluate different numbers of selected features
- Use a suitable regression feature-selection method
- Calculate RMSE for each number of features
- Calculate R-squared for each number of features

Display two side-by-side charts:

- RMSE by Number of Features
- R-squared by Number of Features

Also show:

- selected feature names
- best number of features
- lowest RMSE
- highest R-squared

IMPORTANT:

- These results must NOT be hard-coded from 50_Startups.
- Loading a different CSV must recalculate the results and update both charts.
- The existing 50 Startups dataset should still work as the default example.
- Boston Housing should also work through the same reusable workflow.
- If a dataset is unsuitable for regression, show a clear error message instead of crashing.

Architecture:
The current site is deployed on Vercel.
If Python/scikit-learn is required, create a Vercel serverless API endpoint for the analysis and let the frontend send the CSV URL and selected target column to that endpoint.

Reuse the existing sklearn / CRISP-DM workflow as much as possible.

Test the finished application with at least:

1. 50_Startups.csv
2. A second different regression CSV dataset

Verify that the two datasets produce different model metrics and different feature-selection charts.

Then deploy the finished version to Vercel production and provide the final URL.

這次有一個很重要的技術點：你原本做的是**靜態網頁**，HTML/JavaScript 本身不能直接完整執行你原本的 `scikit-learn main.py` 。

所以如果老師真的要求「換 CSV 後重新跑 sklearn 然後圖也跟著變」，最穩定的方式是：

Vercel 網頁前端
→ 把 CSV URL + target 傳給
Python API
→ `scikit-learn` 分析
→ 回傳 RMSE、R^2、feature selection、prediction results
→ 網頁 JavaScript 即時重畫兩張圖。

而且我會建議一定要增加 **Target 欄位選擇器**。因為「任意 CSV」時程式不可能可靠地猜哪一欄是你要預測的 Y。例如 50 Startups 是 `Profit` ，其他資料可能是 `MEDV` 、 `price` 、 `sales` 等。

所以老師這個加分題如果完整做到，實際上會比目前版本進階不少。你現在的「可輸入 CSV」是第一步，但還要讓它**真的重新分析與生成兩張圖**才完整。

你說：

我要給你一個老師給的作業是第二個作業 先了解我等等船的作業目標之後 跟我說如何在antigravity上操作 此作業跟上一個作業類似

ChatGPT 說：

可以，你把老師的**第二次作業要求**傳給我，我會先完整了解內容，再教你如何使用 **Google Antigravity** 完成。

因為這次作業跟上一次的 **50 Startups / Boston Housing + CRISP-DM** 類似，我會盡量沿用上次的操作方式，讓你不用重新學一套流程。

我會依照以下順序協助你：

1. **分析老師的作業要求**：逐項確認必做內容、加分題、需要產出的圖表，以及繳交方式。
2. **與上次作業比較**：哪些程式和 CRISP-DM workflow 可以沿用，哪些需要修改。
3. **提供 Antigravity 指令**：寫成完整英文 Prompt，讓你可以直接複製貼上。
4. **確認實際執行結果**：協助檢查程式、模型評估數據與圖表是否符合老師要求。
5. **GitHub 上傳**：協助將第二次作業更新至 GitHub。
6. **Vercel 網頁部署**：如果老師要求網頁展示，我會教你部署並測試功能。

這次我會特別注意三件事：

- 不只讓 Antigravity 產生程式，還要**實際執行並驗證**。
- 如果老師要求更換 CSV 後重新生成圖表，就必須確實做到動態分析，而不是只預覽資料。
- 不會在尚未確認老師要求前，自行加入不必要的功能。

你現在直接傳老師第二次作業的截圖、PDF 或文字即可。

我會先幫你整理「老師到底要我們做什麼」，再提供一步一步的 Antigravity 操作教學。

你說：

一、作業目標：
本次作業延伸自 HW1，目標是讓同學能夠以實際資料集進行「多元線性回歸 (Multiple Linear Regression)」的完整分析，並遵循 CRISP-DM 流程完成從資料理解、建模到評估的全過程。

二、作業內容：
1. 資料來源
至 Kaggle 選擇一個具有 10 至 20 個特徵 (features) 的公開資料集。
類型不限（可為房價預測、醫療、車輛效能等主題）。
請明確標示資料集來源與連結。
2. 分析任務
使用線性回歸 (Linear Regression) 模型進行預測。
可嘗試單純線性回歸、多元線性回歸或 Auto Regression。
必須執行 **特徵選擇 (Feature Selection)** 與 **模型評估 (Model Evaluation)**。
結果部分需包含請提供預測圖(加上信賴區間或預測區間)
3. CRISP-DM 流程說明
Business Understanding
Data Understanding
Data Preparation
Modeling

Evaluation
Deployment

4. AI協助要求
所有與 ChatGPT 的對話請以 pdfCrowd 或其他方式須**匯出為 PDF**
請**使用 NotebookLM** 對網路上同主題的解法進行研究，並撰寫一份 100 字以上的摘要，放入報告中。
請在報告中明確標示「GPT 輔助內容」與「NotebookLM 摘要」
5. 繳交內容
主程式：7114056XXX_hw2.py/.ipynb
報告檔： PDF，需包含以下內容：
按照 CRISP-DM 說明的分析流程
GPT 對話過程（pdfCrowd 匯出）
NotebookLM 研究摘要
網路上主流或更優解法之比較與說明
以上檔案與資料夾**請壓縮為學號命名的一個zip**（例如 7114056XXX_hw2.zip）上傳。

(optional) 若上傳至 GitHub，或是以colab撰寫，需提供連結，並在 README.md 中整理流程與成果。

三、評分標準
文件說明（50%）
CRISP-DM 流程完整且邏輯清楚（25%）
包含 GPT 對話與 NotebookLM 摘要（15%）
有明確說明資料集來源與研究脈絡（10%）
 結果呈現（50%）
模型正確可執行，具特徵選擇與評估（25%）
結果合理、美觀且具有說服力（15%）
呈現出Kaggle名次(若有)/預測結果評估(預測圖、評估指標)（10%）

處理時間為 55s

ChatGPT 說：

我已經看完老師的 **HW2 作業要求**。這次確實可以沿用你上次使用 Antigravity 完成 50 Startups 的經驗，但有幾個重要差別。

這次的重點不是做出很多模型，而是使用一份具有 10～20 個特徵的 Kaggle 資料集，完成多元線性回歸、特徵選擇、模型評估，以及包含信賴區間或預測區間的圖表。

而且老師這次將 **文件說明占比提高到 50%**，所以只有程式成功執行還不夠。

一、HW1 與 HW2 的差異

項目	上次 HW1	這次 HW2
資料集	50 Startups / Boston Housing	Kaggle 自選
特徵數量	沒有特別限制	10 ~ 20 個
模型	Multiple Linear Regression	Linear Regression 為核心
CRISP-DM	有	必須完整說明六階段
Feature Selection	已做過	必做
RMSE、MAE、R^2	已做過	必做模型評估
模型比較圖	已做過	可以沿用做法
特徵選擇比較圖	已做過	可以沿用做法
預測圖 + 信賴 / 預測區間	先前未確認	必做
NotebookLM 摘要	沒有	必做，100 字以上
GPT 對話 PDF	沒有	必做
PDF 分析報告	未要求此格式	必做
GitHub / Colab	上次有使用	選做，使用時須提供連結
Vercel	上次有部署	這次沒有要求

二、老師評分重點

文件說明與結果呈現各占一半

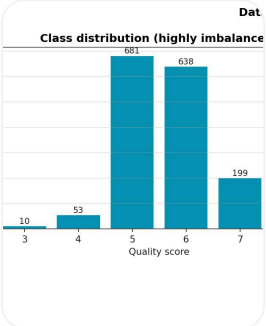
文件說明 **50%**

結果呈現 **50%**

細項：CRISP-DM 25%、GPT / NotebookLM 15%、資料來源 10%、程式及評估 25%、成果品質 15%、Kaggle 名次（若有）或預測評估 10%。

這也代表上次我們強調的「多模型比較、網頁部署」不是 HW2 最優先的事情。這次應先確保 **回歸模型正確、研究說明完整、報告能交。**

三、我建議 HW2 使用的 Kaggle 資料集



Red Wine Quality（紅酒品質預測）

Kaggle 公開資料集 · 1,599 筆資料 · 11 個預測特徵

使用酸度、糖分、酒精濃度、pH 等化學指標，預測紅酒品質分數 `quality`。

查看 Kaggle 資料集 ↗

這份資料很適合你的作業，原因是：

- **確實符合 10～20 個特徵**：有 11 個輸入特徵，不把目標 `quality` 算進去。
- 數值特徵為主，適合 `sklearn.linear_model.LinearRegression`。
- 可以使用 `SelectKBest` 或 `RFE` 做特徵選擇。
- 有現成研究和 Kaggle 解法可供 NotebookLM 比較。
- 可以製作預測值與實際值比較圖，以及 95% 預測區間。

這份資料的來源與欄位數已經查到，並非猜測。 k kaggle.com +1

要注意的是，紅酒品質分數本身是離散評分，但這次可依老師要求，將它視為數值目標進行回歸分析，並在報告中說明限制。

四、這次 Antigravity 的操作順序

1 **建立新專案 HW2**

不要直接覆蓋原本 `1014` 的 HW1 專案。建立新的 `HW2` 資料夾，放入 Kaggle CSV。

2 **貼入 Antigravity Prompt**

讓 Agent 按照 CRISP-DM 建立 Python 程式、執行特徵選擇與回歸分析。

3 **驗證分析結果**

檢查 MAE、MSE、RMSE、R^2、特徵選擇結果和預測圖。尤其確認圖上的 95% 預測區間有實際計算，不是單純畫兩條裝飾線。

4 **使用 NotebookLM 研究**

找相關研究來源，放進 NotebookLM，產生並核對 100 字以上中文摘要。

5 **製作 PDF 報告與 GPT 對話紀錄**

整理 CRISP-DM 六階段、實驗結果、模型比較、NotebookLM 摘要與 ChatGPT 對話紀錄。

6 **整理 ZIP 繳交**

用自己的實際學號命名 `.py` 、PDF 報告與 ZIP。GitHub 和 Vercel 可以之後再做。

五、第一階段：直接貼給 Antigravity 的指令

先在 Kaggle 下載紅酒品質 CSV，放到新建的 `HW2` 資料夾。接著開啟 Antigravity 的 Agent 對話框，把以下整段貼進去。

寫作

I need to complete Homework 2 for a university Data Mining course using Python, scikit-learn, and the CRISP-DM methodology.

This homework is an extension of my previous 50 Startups regression project. Reuse the same general CRISP-DM workflow, but create an independent HW2 project.

Dataset:
Red Wine Quality Dataset from Kaggle:
https://www.kaggle.com/datasets/uciml/red-wine-quality-cortez-et-al-2009

Use the actual CSV file in my project folder. Do not fabricate or simulate dataset observations.

Requirements:

1. Business Understanding
- Explain the objective of predicting red wine quality.
- Use Multiple Linear Regression as the primary model.
- Clearly define the 11 input features and the target variable `quality`.

2. Data Understanding
- Load the CSV and check the delimiter.
- Inspect shape, columns, missing values, duplicates, data types and descriptive statistics.
- Verify there are 11 original predictive features.
- Generate correlation heatmap and exploratory scatter plots.

3. Data Preparation
- Separate X and y.
- Use an 80/20 train/test split with random_state=42.
- Handle missing values if present.
- Build preprocessing with sklearn Pipeline.
- Avoid data leakage.
- Apply feature selection using SelectKBest or RFE.
- Perform feature selection only within training data or cross-validation folds.

4. Modeling
- Train Multiple Linear Regression as the required main model.
- Also compare Ridge and Random Forest as optional reference models.
- Do not replace the required linear regression analysis with another model.
- Perform 5-fold cross-validation on training data.
- Record the selected features and regression coefficients.

5. Evaluation
Calculate:
- MAE
- MSE
- RMSE
- R-squared
- Training R-squared
- Testing R-squared

Create and save:
1. actual_vs_predicted.png
2. feature_selection_metrics.png
3. model_comparison.png
4. residual_plot.png
5. correlation_heatmap.png
6. learning_curve.png

Important new homework requirement:
Create a prediction plot with a statistically justified 95% prediction interval.

Save as:

prediction_interval.png

Use a suitable method, such as split conformal prediction with a separate calibration set.

Clearly label the interval as a prediction interval rather than a confidence interval. Explain its assumptions and limitations, and report empirical test coverage.

Do not create arbitrary confidence bands or hard-code evaluation metrics.

6. Deployment
- Save the trained preprocessing and linear regression pipeline.
- Verify the saved model reloads and predicts successfully.

7. Reporting Outputs
- Save evaluation metrics in JSON or CSV.
- Save feature-selection results in CSV.
- Create a clear Markdown report draft following all six CRISP-DM stages.
- Include sections named "GPT 輔助內容" and "NotebookLM 摘要".
- Leave the NotebookLM summary as a placeholder until actual NotebookLM research is supplied.
- Include the original Kaggle dataset URL.
- Include a comparison of linear regression with alternative methods based on actual results.
- Do not invent NotebookLM findings, citations, Kaggle rankings or results.

8. Execution
- Create a Python source file named hw2.py initially.
- Run the entire project.
- Fix execution errors.
- Verify every generated graph and results file exists.
- Do not modify my previous HW1 project.
- Show me the final evaluation metrics and explain the findings in Traditional Chinese.

I will rename the final Python file using my actual student ID before submission.

這段我特別幫你加入老師最重視、但 HW1 沒有明確要求的 **95% Prediction Interval**，以及避免特徵選擇造成資料洩漏的設定。

六、最後繳交的資料夾應該長這樣

7114056XXX_hw2/

- 7114056XXX_hw2.py
- winequality-red.csv
- 7114056XXX_hw2_report.pdf
- GPT_conversation.pdf
- figures/（所有分析圖表）
- results/（評估數據與特徵選擇結果）
- README.md（若上傳 GitHub）

7114056XXX_hw2.zip

7114056XXX 是老師提供的格式範例，要替換成你的實際學號。

七、我建議這次先不要急著部署 Vercel

因為老師 HW2 明確要求的是：

Python 程式 + PDF 報告 + GPT 對話 PDF + NotebookLM 摘要 + ZIP。

所以這次我們先把 Antigravity 分析做好，接著再製作 NotebookLM 摘要與 PDF 報告。至於 GitHub、Vercel，等必要項目完成後再處理，才不會花很多時間做不計分的部分。

你現在第一步就是下載 Kaggle 的 Red Wine Quality CSV，建立 HW2 資料夾，再把上面的指令貼給 Antigravity。

跑完後把 Antigravity 畫面傳給我，我會依老師的評分標準幫你核對每一項，包括那張最重要的「含 95% 預測區間的預測圖」。

你說：

所以你選ㄉ題目是啥

ChatGPT 說：

我幫你選的 HW2 題目是：

紅酒品質預測（Red Wine Quality Prediction）

使用多元線性回歸（Multiple Linear Regression）預測紅酒品質分數。

- **資料來源：** Kaggle
- **資料集：** Red Wine Quality
- **資料筆數：** 1,599 筆
- **特徵數量：** 11 個
- **預測目標（Y）：** 紅酒品質分數 `quality`
- **使用模型：** Multiple Linear Regression
- **分析方法：** CRISP-DM、Feature Selection、Model Evaluation



Kaggle 資料集連結：
Red Wine Quality Dataset ↗

為什麼選這個題目？

1. 符合老師要求的 **10～20 個特徵**。
2. 11 個特徵都是數值型態，資料前處理比較簡單。
3. 適合做特徵選擇、線性回歸、RMSE、R^2 評估。
4. 可以製作老師要求的預測圖與 95% 預測區間。
5. 和上次 50 Startups 作業的分析流程很接近。

建議正式作業名稱：

「基於 CRISP-DM 流程之紅酒品質多元線性回歸預測與特徵選擇分析」

英文：*Red Wine Quality Prediction Using Multiple Linear Regression and Feature Selection Based on CRISP-DM*

這個題目比上次 50 Startups 多了特徵，但整體難度仍然適中，很適合用 Antigravity 完成。

你說：



我怎麼知道已經做完ㄌ

ChatGPT 說：

從你這張 Antigravity 截圖來看，**HW2 的程式與圖表大致都已產生，但還不能確認整份作業已經 100% 完成。**

因為這次老師不只要求 Python 程式，還要求 PDF 報告、NotebookLM 摘要及 GPT 對話紀錄。

一、目前完成狀況

我依照你截圖左邊的檔案，對照老師要求：

老師要求	目前狀態
Kaggle 資料集（10～20 特徵）	✅ 有資料檔
Python 主程式	✅ `hw2.py` 已建立
CRISP-DM 六階段	⚠️ 需確認報告內容
Feature Selection	✅ 已產生圖表與 CSV
Model Evaluation	✅ 已產生 JSON
模型比較圖	✅ 已產生
Learning Curve	✅ 已產生
預測圖＋95% 預測區間	✅ 已產生圖片
NotebookLM 100 字摘要	⚠️ 尚未確認
GPT 對話 PDF	❌ 截圖未看到
PDF 分析報告	❌ 截圖未看到
學號命名 ZIP	❌ 截圖未看到

注意：有產生檔案不等於分析一定正確，還需要驗證執行結果與圖表內容。

二、現在先讓 Antigravity 自動檢查

你直接複製以下指令，貼到右下角的 **Ask anything** 對話框：

寫作

Please perform a complete final verification of my HW2 project against the instructor's requirements.

1. Run hw2.py from start to finish.
2. Check whether the program executes successfully without errors.
3. Verify the Kaggle dataset has 10–20 predictive features.
4. Verify all six CRISP-DM stages are documented.
5. Verify feature selection and all regression evaluation metrics.
6. Verify that prediction_interval.png contains a statistically valid 95% confidence interval or prediction interval, with the method explained.
7. Check that all generated PNG, CSV and JSON files contain valid results.
8. Check the editor warning/problem shown in hw2.py and explain whether it affects execution.
9. Verify that HW2_Report.md contains the required analysis and model comparison.
10. Check whether the NotebookLM summary is genuine and contains at least 100 Chinese characters. Do not fabricate NotebookLM research.
11. Check whether the ChatGPT conversation PDF and final report PDF exist.
12. Check whether the files are ready to be renamed using my student ID and compressed into a ZIP.

Please generate a final checklist in Traditional Chinese using:
- PASS: verified
- FAIL: missing or incorrect
- NOT VERIFIED: cannot confirm

Do not claim that the assignment is fully complete unless every mandatory requirement is satisfied.

Do not overwrite existing results unless fixing a confirmed issue.

三、接下來要完成的三件重要事情

NotebookLM 研究摘要

到 NotebookLM ↗ 匯入同主題的研究資料，撰寫至少 100 字的中文摘要，並放入 `HW2_Report.md` 。這部分不能只讓 Antigravity 假裝完成。

PDF 報告與 GPT 對話紀錄

目前有 `HW2_Report.md` ，但老師要求正式 PDF。需要將報告轉 PDF，並另外匯出 ChatGPT 對話紀錄。

ZIP 繳交

確認程式與報告無誤後，將所有必交檔案整理好，以你的學號命名 ZIP。

四、我的建議

你現在先不要急著交作業。

你這張畫面顯示 Antigravity 已經建立了大量分析成果，這是很好的進度。但老師這次文件占 50%，所以 NotebookLM 摘要與 PDF 報告同樣重要。

先把我上面的英文指令貼給 Antigravity，讓它產生最後的檢查結果，再把結果截圖給我。

我就能幫你判斷：**哪些已經確實做完、哪些需要補，以及目前是否符合老師 HW2 的繳交要求。**

你說：

我看不到結果嗎 譬如整個作業的檔案

ChatGPT 說：

可以！其實你剛剛的 Antigravity 畫面，**左邊就已經有整份 HW2 作業產生的檔案了**，只是目前你看到的是程式碼，還不是完整的分析成果。

我教你怎麼看。

一、如何查看整份作業的結果？

在你剛剛 Antigravity 的左側 Explorer（檔案總管），可以看到這些檔案：

檔案名稱	功能	怎麼查看
`HW2_Report.md`	完整分析報告草稿	點兩下，或右鍵選 Open Preview

`README.md`	專案介紹	點開
`hw2.py`	Python 主程式	點開查看程式碼
`actual_vs_predicted.png`	實際值與預測值比較圖	雙擊
`prediction_interval.png`	95% 預測區間圖	雙擊
`feature_selection_metrics.png`	特徵選擇比較圖	雙擊
`model_comparison.png`	不同模型比較圖	雙擊
`learning_curve.png`	模型學習曲線	雙擊
`evaluation_metrics.json`	MAE、MSE、RMSE、R^2 等結果	點開

二、我最推薦你現在先看這個

HW2_Report.md

這是 Antigravity 幫你產生的作業報告草稿。

你可以在 Antigravity：

1. 在左邊找到 `HW2_Report.md` 。
2. 點開這個檔案。
3. 按鍵盤 **Ctrl + Shift + V**，開啟 Markdown 預覽。

這樣就能用排版好的形式查看整份作業報告，而不是密密麻麻的程式碼。

三、如果你想要一份完整、可以直接看的作業成果

你可以要求 Antigravity 幫你做一份有文字、圖表、模型評估結果的整合報告。

直接把以下指令貼到右邊 Agent 對話框：

寫作

Please organize my entire HW2 project into one complete, easy-to-read report.

Requirements:

1. Use the existing HW2_Report.md and actual analysis results.
2. Include all six CRISP-DM stages.

3. Insert all relevant generated figures:
   - exploratory_scatter_plots.png
   - correlation_heatmap.png
   - feature_selection_metrics.png
   - model_comparison.png
   - actual_vs_predicted.png
   - prediction_interval.png
   - residual_plot.png
   - learning_curve.png
4. Include actual MAE, MSE, RMSE, R^2, feature-selection results, and model comparison results.
5. Explain the findings in Traditional Chinese.
6. Do not invent any metrics or analysis results.
7. Clearly mark sections for GPT-assisted content and the NotebookLM summary.
8. Do not fabricate a NotebookLM summary. Leave a clearly marked placeholder if I have not provided one.
9. Generate a complete HTML report named HW2_Final_Report.html with all images and proper formatting.
10. Open the HTML report for preview.
11. Also generate HW2_Final_Report.pdf if PDF export tools are available, and verify the output.

Keep the original Python code, CSV dataset, charts, and analysis files unchanged.

這樣做完，你就可以用瀏覽器開啟 `HW2_Final_Report.html` ，一次看到整份作業的分析文字、圖表與結果。

不過要注意： 這份整合報告仍需要加入你實際完成的 NotebookLM 摘要及 GPT 對話 PDF，才能符合老師所有的繳交要求。

你現在可以先試著打開 `HW2_Report.md` **並按 Ctrl + Shift + V**，就能先看到 Antigravity 已經做好的報告內容。

你說：

(optional) 若上傳至 GitHub，或是以colab撰寫，需提供連結，並在 README.md 中整理流程與成果。
老師有寫這個代表我能放在github上ㄇ

ChatGPT 說：

對！✅ **老師明確允許你把 HW2 作業放到 GitHub 上。**

老師寫的：

> (optional) 若上傳至 GitHub，或是以 Colab 撰寫，需提供連結，並在 README.md 中整理流程與成果。

意思是：

- **GitHub 是選擇性項目**，不是強制要求。
- 如果你選擇上傳 GitHub，就必須提供 GitHub 網址。
- GitHub 的 `README.md` 必須整理 CRISP-DM 分析流程及研究成果。

我建議你跟 HW1 一樣放到 GitHub

不過這次建議另外建立一個 `HW2` Repository，不要跟上次的 `1014` 混在一起。

你可以直接在 Antigravity 輸入：

寫作

```
Please prepare my HW2 Red Wine Quality Regression project for GitHub.

Requirements:

1. Create a professional README.md in Traditional Chinese.
2. Include:
   - Project title
   - Kaggle dataset source URL
   - Research objective
   - All six CRISP-DM stages
   - Feature selection results
   - Model evaluation metrics (MAE, MSE, RMSE, R²)
   - Prediction results with 95% prediction interval
   - Model comparison results
   - Conclusions
3. Embed the existing analysis images in README.md so they are visible directly on GitHub.
4. Use only actual results from this project.
5. Keep the existing Python code and dataset unchanged.
```

6. Prepare the project for uploading to a NEW GitHub repository.
7. Do not push anything until I provide the repository URL.

但要特別注意

GitHub 不能完全取代老師要求的 ZIP 繳交檔案。

這次老師明確要求：

`學號_hw2.zip`

裡面要有 Python 程式、PDF 報告、GPT 對話 PDF、NotebookLM 摘要等內容。

所以最適合的方式是：

GitHub：展示程式、流程、圖表與分析成果。

ZIP：按照老師規定，正式上傳作業平台。

這樣既符合老師要求，也能延續你上次 HW1 的 GitHub 操作方式。

你說：

我想放到github https://github.com/john19991225-commits/hw2.git
怎麼下指令

ChatGPT 說：

可以！你這次要把 **HW2 紅酒品質預測作業**上傳到 GitHub：

你的 GitHub Repository

https://github.com/john19991225-commits/hw2 ↗

跟上次 HW1 一樣，你可以直接在 **Antigravity 右側的 Agent 對話框**輸入下面這段指令。

方法一：讓 Antigravity 自動上傳（推薦）

寫作

Please upload my current HW2 Red Wine Quality Regression project to GitHub.

Repository:
https://github.com/john19991225-commits/hw2.git

Requirements:

1. Confirm that the current working directory contains hw2.py, winequality-red.csv, README.md, and all generated analysis charts.
2. Update README.md in Traditional Chinese to include:
   - Project title
   - Kaggle dataset source
   - All six CRISP-DM stages
   - Feature selection results
   - MAE, MSE, RMSE, and R^2
   - Model comparison results
   - 95% prediction interval visualization
   - Conclusions
3. Embed the generated PNG charts in README.md so they appear directly on the GitHub repository homepage.
4. Use only verified results. Do not fabricate metrics.
5. Create a .gitignore to exclude temporary files, caches, local environments, and credentials.
6. Initialize Git if necessary.
7. Configure the remote origin as:
   https://github.com/john19991225-commits/hw2.git
8. Commit the project files with the message:
   "Add HW2 Red Wine Quality Regression Analysis"
9. Push the commit to the main branch.
10. Verify the push succeeded and confirm that the remote main branch contains the latest commit.

Do not delete or overwrite existing remote files or force push.

Return the GitHub repository URL after successful verification.

方法二：自己在 Terminal 輸入 Git 指令

如果你想跟上次一樣自己操作，可以在 Antigravity 上方選：

Terminal → New Terminal

然後輸入：

```
git init
git remote add origin https://github.com/john19991225-commits/hw2.git
git add .
git commit -m "Add HW2 Red Wine Quality Regression Analysis"
git branch -M main
git push -u origin main
```

如果出現 `remote origin already exists`，改成：

```
git remote set-url origin https://github.com/john19991225-commits/hw2.git
```

再執行 `git push -u origin main`。如果遠端已經有其他提交，先不要強制推送，應先確認和整合遠端內容。

上傳後應該看到什麼？

hw2 — GitHub Repository

- hw2.py
- winequality-red.csv
- README.md
- feature_selection_metrics.png
- model_comparison.png
- prediction_interval.png
- evaluation_metrics.json

以上是預期上傳的主要檔案，實際內容以 GitHub 上傳結果為準。

最重要的是 `README.md`。如果圖片已經嵌入 README，老師點開 GitHub 網址，就能直接看到你 HW2 的分析流程、預測結果與圖表，不需要逐一打開圖片。

另外，這次老師還要求 PDF 報告及 ZIP，GitHub 上傳成功後，仍要完成正式作業繳交。

我建議你先使用方法一，把英文指令貼給 Antigravity，完成後把右側 Agent 顯示的結果截圖給我，我幫你確認 GitHub 是否真的上傳成功。

你說：

https://github.com/john19991225-commits/hw2.git進去幫我檢查我作業有無完整

處理時間為 37s

ChatGPT 說：

我已經實際進入你的 GitHub：

HW2 紅酒品質預測 GitHub 專案 ↗

先說結論：**你的程式、資料集、CRISP-DM 分析和主要圖表都已經上傳，而且 README 的內容相當完整；但按照老師正式繳交要求，目前還不能算全部完成。**

尤其是 **NotebookLM 摘要、GPT 對話 PDF、正式 PDF 報告，以及學號命名的 ZIP**，這幾項需要補齊或確認。

一、逐項對照老師要求

老師要求	GitHub 檢查結果
Kaggle 公開資料集	✅ 有來源連結
10～20 個特徵	✅ 11 個
多元線性回歸	✅ 有程式及結果
Feature Selection	✅ 有圖表與 CSV
Model Evaluation	✅ 有評估指標
預測圖	✅ 有 PNG
95% 預測區間	✅ 有圖及方法說明
CRISP-DM 六階段	✅ README 有完整章節
多模型比較	✅ 有 Linear、Ridge、Random Forest
NotebookLM 研究摘要	⚠️ 需確認是否仍為佔位文字
GPT 對話 PDF	❌ GitHub 沒看到
正式報告 PDF	❌ GitHub 沒看到
學號命名主程式	⚠️ 目前仍為 `hw2.py`

學號命名 ZIP	⊗ GitHub 沒看到

以上檔案存在情況是我直接從 GitHub 的 `main` 分支檢查的，並不是只看你先前的 Antigravity 截圖。 john199912... +1

要補充：**老師沒有規定所有正式繳交檔案都必須放在 GitHub**。PDF 和 ZIP 可以留在電腦中，最後上傳作業平台。所以「GitHub 沒看到」不代表你本機一定沒有。

二、目前 GitHub 上的模型成果

你的 README 已記錄以下結果：

模型	Test RMSE ↓	Test R^2 ↑
Multiple Linear Regression	0.6245	0.4032
Ridge Regression	0.6245	0.4032
Random Forest	0.5489	0.5390

特徵選擇部分，README 顯示 **8 個特徵**時的 5-fold CV RMSE 最低，為 `0.6593`。此外，95% 保形預測區間的測試覆蓋率記錄為 `99.06%`（317/320）。

這些是 GitHub 已發表的分析結果；我還在確認報告中的說法和程式實作是否一致。
john199912... +1

三、我確認到一項尚未完成的必做內容

我打開了你的 HW2_Report.md ↗，找到第八章「NotebookLM 摘要」。

目前裡面仍然寫著：

> `[NotebookLM 摘要佔位符號]`

也就是說，**NotebookLM 的實際研究摘要尚未填入報告**。這一點可以明確確認。 GitHub

另外，第七章「GPT 輔助內容」已經有文字說明，但老師要求的**完整 ChatGPT 對話 PDF**是另一項獨立要求，不能只用這一章代替。

四、程式方面的進一步檢查

我也打開了 `hw2.py` 的程式內容，確認其中確實有：

- `train_test_split` 的 80/20 分割
- `SelectKBest` 特徵選擇

- 線性回歸及模型比較
- MAE、MSE、RMSE、R^2 評估
- 使用獨立校準集計算的 Split Conformal Prediction
- 產生 `prediction_interval.png` 的程式

這些不是只在 README 宣稱有做，而是能在 Python 原始碼中找到對應實作。 GitHub +1

不過這次檢查是閱讀 GitHub 原始碼與已上傳結果，**尚未在獨立環境重新執行整支程式**，因此不能把可重現性標示為完全實測通過。

五、建議你現在補的內容

我建議先處理下面三項，順序不要顛倒：

1. **NotebookLM 摘要**：真的使用 NotebookLM 研究紅酒品質預測相關論文，至少撰寫 100 字，貼回報告第八章。
2. **正式 PDF 報告**：把 `HW2_Report.md` 整理成有完整圖片、表格和研究摘要的 PDF。
3. **正式繳交 ZIP**：另外匯出這次 ChatGPT 對話 PDF，把程式、資料、報告和對話紀錄放進學號命名的 ZIP。

之後也建議補上 `requirements.txt` ，讓老師可以安裝所需套件並重現程式。

可以直接貼給 Antigravity 的指令

寫作

```
Please review my existing HW2 project and prepare it for final submission.

The GitHub repository is:
https://github.com/john19991225-commits/hw2

Important:
- Do not change the existing verified regression results unnecessarily.
- Do not fabricate NotebookLM research or GPT conversations.
- Do not push the final submission ZIP or personal conversation PDF to a public GitHub repository.

Tasks:
1. Check all mandatory requirements against the instructor's HW2 assignment.
2. Preserve the current CRISP-DM analysis, figures and model evaluations.
```

3. Prepare HW2_Report.md for insertion of my actual NotebookLM research summary, with at least 100 Chinese characters.
4. Generate a professionally formatted PDF report with all existing figures and tables, after the summary is inserted.
5. Create requirements.txt containing the dependencies needed to reproduce hw2.py.
6. Run hw2.py again and verify successful execution.
7. Check the 95% prediction interval implementation and reported empirical coverage.
8. Prepare a local submission folder with the Python program, original dataset, report PDF, figures and results.
9. Reserve a place for my ChatGPT conversation PDF.
10. Once all required documents are present, create a ZIP using my actual student ID.
11. Report missing requirements clearly in Traditional Chinese.

Do not mark the submission as complete until the NotebookLM summary and ChatGPT PDF have actually been included.

最後判斷： 你 GitHub 的**分析主體已相當完整**，包含老師要的多元線性回歸、特徵選擇、模型評估及預測區間；真正需要優先補的是研究摘要與正式繳交文件。

我不會在文件還缺少必要內容時說你已經可以直接交。

你說：

你再去看看可不可以ㄌ

ChatGPT 說：

我剛剛重新打開你的 HW2 GitHub ↗，也重新讀取了最新的 `README.md` 和 `HW2_Report.md`。

目前仍然還差幾項，尚未達到老師要求的完整繳交狀態。

最新檢查結果（2026/10/19）

檢查項目	狀態
Kaggle 資料集、11 個特徵	✅ 已完成
多元線性回歸	✅ 已有成果
CRISP-DM 六階段	✅ 已有完整說明
Feature Selection	✅ 已完成

MAE、MSE、RMSE、R^2	✅ 已有數據
模型比較與預測圖	✅ 已上傳
95% 預測區間圖	✅ 已上傳
NotebookLM 100 字以上摘要	❌ 仍未填入
GPT 對話 PDF	⚠️ GitHub 未看到
正式 PDF 報告	⚠️ GitHub 未看到
學號命名 ZIP	⚠️ GitHub 未看到

GitHub 目前列出 19 個檔案，仍是 1 筆 commit。 john19991225-...

最主要的問題：NotebookLM 仍未完成

我重新打開報告，發現第八章依然寫著：

`[NotebookLM 摘要佔位符號]`

而不是正式的研究摘要。 GitHub

另外還發現兩個小地方：

- 報告的「學生姓名」仍是 **[待填寫]**。
- 程式在 GitHub 上仍叫 `hw2.py`，正式繳交時應改成老師指定的學號格式。

現在應該怎麼做？

我建議你**先完成 NotebookLM 這一項**，因為老師有明確給這部分評分，而且不能用 ChatGPT 假裝是 NotebookLM 的研究結果。

完成後，把 NotebookLM 真正產生的摘要貼進 `HW2_Report.md`，再重新產生 PDF 報告。

目前的判斷：分析成果大致齊全，但正式繳交文件還需要補強。 我還無法確認你電腦本機是否已經有 PDF 和 ZIP，因此不會直接判定那些檔案完全沒做。

你說：



是要改這個py ㄅㄇ

ChatGPT 說：

對，你截圖中的 `hw2.py` **就是這次作業的 Python 主程式**。

但你要修改哪個檔案，要看修改的內容：

檔案	用途	需要修改嗎？
`hw2.py`	多元線性回歸、特徵選擇、模型評估	程式能正常執行就不用修改內容
`HW2_Report.md`	CRISP-DM 分析報告	需要補 NotebookLM 摘要
`README.md`	GitHub 首頁的作業介紹	可以補充最新成果

老師要求主程式以學號命名

例如你的學號是 `7115056206` ，就應該改成：

`7115056206_hw2.py`

這只是命名範例，請以你的實際學號為準。

你現在應該先做的事

第一步： 點開 `HW2_Report.md` ，找到第八章「NotebookLM 摘要」，填入實際研究內容。

第二步： 確認 `hw2.py` 能正常執行，再依學號重新命名。

第三步： 完成 PDF 報告、GPT 對話 PDF 和 ZIP。

另外，你截圖中 `hw2.py` 右邊有紅色的 **1**，代表編輯器偵測到一項問題或警告，但目前無法確認是否會影響執行。

所以你不用重新做整份作業，也不需要重寫 Python 程式。 目前主要是補齊報告內容和正式繳交檔案。

你說：

可是我的github沒變ㄟ

ChatGPT 說：

這是因為 **你在 Antigravity 修改檔案後，GitHub 不會自動更新**，還需要重新執行 Git Push。

你現在可以直接在 **Antigravity 右側的 Agent 對話框**貼上這段指令：

寫作

Please synchronize my current HW2 project with GitHub.

Repository:
https://github.com/john19991225-commits/hw2.git

Steps:
1. Check the current Git branch and remote repository.
2. Check git status to identify all modified and new files.
3. Save the latest README.md, HW2_Report.md, Python code, and generated analysis results.
4. Stage the updated project files, excluding temporary files, credentials, and personal conversation records.
5. Commit with the message "Update HW2 report and analysis".
6. Push to the main branch.
7. Verify that the latest commit appears on GitHub.
8. Confirm whether the NotebookLM summary is actually present in HW2_Report.md.

Do not force push or overwrite unrelated remote changes.

Show me the Git commit ID and whether the push succeeded.

如果你想自己輸入指令

在 Antigravity 上方選擇 **Terminal → New Terminal**，確認目前位於 HW2 專案資料夾，輸入：

```
git status
git add README.md HW2_Report.md hw2.py
git commit -m "Update HW2 report"
git push origin main
```

如果你還修改了圖片或評估結果，要把相關檔案一起加入提交。

怎麼確認真的成功？

1. 打開你的 GitHub HW2 專案 ↗。
2. 重新整理網頁（Ctrl + F5）。

3. 查看最新的 Commit 是否變成 `Update HW2 report and analysis`（如果你使用上面的 Agent 指令）。
4. 點開 `HW2_Report.md`，確認修改的內容已經顯示。

提醒： 如果你只是把 `hw2.py` 改名，或修改 `HW2_Report.md`，但沒有 `git commit` 和 `git push`，GitHub 就不會改變。

你可以先把英文指令貼給 Antigravity。完成後將右側的執行結果截圖給我，我就能幫你確認是否成功同步。

你說：

NotebookLM 研究摘要
這怎麼做

處理時間為 7s

ChatGPT 說：

這個很簡單，我教你一步一步操作。你這次 HW2 的主題是 **紅酒品質預測（Red Wine Quality Prediction）**，老師要求使用 NotebookLM 研究網路上同主題的解法，並把 **100 字以上的研究摘要**放進作業報告。

你不需要重新做 Python 程式，只要用 NotebookLM 研究其他人的方法，再跟你自己的模型比較即可。

第一步：進入 NotebookLM

Google NotebookLM

Google 提供的 AI 研究整理工具，可以根據加入的論文與網頁生成摘要和引用。

blog.google

開啟 NotebookLM

進去後，使用 Google 帳號登入，再選擇「建立新筆記本」（Create new notebook）。

筆記本名稱建議填：

```
HW2 - Red Wine Quality Prediction
```

第二步：加入研究資料

我幫你找到兩個可以直接使用的研究來源。

來源一：Wine Quality Dataset — UCI

這是紅酒品質資料集的原始來源，介紹變數、品質評分及研究背景。 UCI Machine Le...

開啟 UCI 資料集介紹 ↗

來源二：Cortez et al. (2009) 研究論文

Modeling wine preferences by data mining from physicochemical properties

這篇研究比較了多元迴歸、神經網路與支援向量機等方法，適合拿來比較你的 sklearn 分析。 ScienceDirect

開啟研究論文 PDF ↗

在 NotebookLM 裡：

1. 點「新增來源」（Add sources）。
2. 選擇網站連結，貼上第一個網址。
3. 第二篇可以下載 PDF 後，上傳到 NotebookLM。
4. 確認兩份資料都成功加入來源清單。

如果網站無法擷取，就使用 PDF 或 NotebookLM 的 Discover sources 搜尋功能。 blog.google

第三步：讓 NotebookLM 產生研究摘要

來源加入完成後，在 NotebookLM 中間的對話框貼上以下指令：

寫作

請根據我上傳的紅酒品質預測（Red Wine Quality Prediction）相關研究資料，撰寫一份 200～300 字的繁體中文研究摘要，作為大學資料探勘課程 HW2 報告的「NotebookLM 研究摘要」。

請包含：

1. 原始研究的目標及使用的資料集。
2. 研究使用哪些回歸或機器學習模型。
3. 多元線性回歸與其他模型在預測表現上的差異。
4. 哪些紅酒化學特徵可能對品質預測有幫助，以及研究如何判斷特徵的重要性。
5. 這些研究對我使用 sklearn、Multiple Linear Regression、Feature Selection 進行紅酒品質預測，有哪些參考價值。

請使用正式但容易理解的繁體中文。

所有研究結論都必須根據已加入的來源，附上來源引用。若文獻沒有明確提供某項結果，請標示無法確認，不要自行編造模型數值、特徵排名或研究結論。

最後加入一小段「與本次 HW2 實作方法的比較」，只比較方法，不要假設已知我的實驗數值。

這樣 NotebookLM 就會根據你匯入的資料生成摘要。

第四步：把摘要放進 Antigravity

回到你之前的 Antigravity 專案。

打開：

`HW2_Report.md`

找到：

`八、NotebookLM 摘要`

把原本的：

`[NotebookLM 摘要佔位符號]`

換成 NotebookLM 實際產生、經你核對的摘要。

你也可以將摘要直接貼給 Antigravity，並下這個指令：

寫作

Please update Section 8, "NotebookLM 摘要", in HW2_Report.md using the actual NotebookLM summary I provide below.

Requirements:
- Preserve the original research findings and source citations.
- Use Traditional Chinese.
- Keep the summary at least 100 Chinese characters.
- Do not invent research findings or references.
- Do not modify any model evaluation results.
- Update README.md to indicate that the NotebookLM literature review has been completed.

Here is my actual NotebookLM summary:

[Paste my NotebookLM summary here]

第五步：更新 GitHub

完成報告後，回到 Antigravity 的 Terminal：

```
git add HW2_Report.md README.md
git commit -m "Add NotebookLM literature review"
git push origin main
```

接著打開 你的 HW2 GitHub ↗ ，確認摘要已經更新。

提醒： 老師要求的不只是使用 NotebookLM，而是要將真正的研究摘要放進最終 PDF 報告。因此 GitHub 更新後，最後產生的 PDF 也要包含這段內容。

你現在可以先完成**第一步到第三步**。等 NotebookLM 產生研究摘要，你直接把文字或截圖傳給我，我會幫你核對是否符合老師的「100 字以上、網路研究、主流或更優解法比較」要求。

你說：

來源2打不開

ChatGPT 說：

沒關係，我重新幫你找了一個可以替代的來源，而且是**同一篇 2009 年的原始研究論文**。

來源二：紅酒品質預測研究論文

論文名稱： *Modeling wine preferences by data mining from physicochemical properties*

作者：Paulo Cortez 等人（2009）

開啟 MIT 論文 PDF

我已確認搜尋系統能讀取這份 PDF 的論文內容。 projects.csail.mi...

如果 PDF 還是無法開啟，也可以使用期刊頁面：

ScienceDirect 原始論文介紹 ↗ 。 ScienceDirect

接下來這樣操作

1. 點上方的 MIT 論文 PDF 連結。
2. 開啟後下載 PDF。
3. 回到 NotebookLM，點「新增來源」。
4. 選擇「上傳檔案」，將 PDF 加入。
5. 加入成功後，使用我前面給你的摘要指令。

這篇論文會比較**多元迴歸、神經網路和支援向量機**，很適合用來完成老師要求的「網路上主流或更優解法之比較與說明」。

如果你現在已經把 UCI 資料集和這篇 PDF 都加入 NotebookLM，就可以直接開始生成 100 字以上的研究摘要。

你說：



這啥意思

ChatGPT 說：

這個畫面的意思是：**Antigravity 已經幫你準備好 HW2 的作業打包程式，現在只差把 ChatGPT 對話紀錄轉成 PDF，才能產生最後繳交的 ZIP 檔。**

你只需要完成兩個步驟。

第一步：把 ChatGPT 對話存成 PDF

就是我們現在這個聊天紀錄。

1. 在電腦瀏覽器開啟這段 ChatGPT 對話。
2. 按鍵盤 **Ctrl + P**（列印）。
3. 印表機選擇「另存為 PDF」或「Microsoft Print to PDF」。
4. 將檔案儲存為：

```
7115056206_ChatGPT_Conversation.pdf
```

5. 將 PDF 放到你的 Antigravity 專案資料夾：

C:\Users\john1\Desktop\物聯網\1015

7115056206_ChatGPT_Conversation.pdf

注意：列印前檢查 PDF 是否包含老師需要的完整對話紀錄。有些瀏覽器不一定會列印所有聊天訊息，必要時可使用老師提到的 pdfCrowd 或其他匯出方式。

第二步：讓 Antigravity 自動打包

把 PDF 放進資料夾後，回到 Antigravity。

點上方：

Terminal → New Terminal

輸入：

```
python prepare_submission.py
```

然後按 Enter。

根據畫面顯示，這個程式會檢查作業檔案並嘗試產生：

7115056206.zip

但要注意：目前我只看到了 Antigravity 的說明，**還沒有實際檢查 `prepare_submission.py` 的內容，也無法確認它執行後一定成功**。

第三步：檢查 ZIP 是否完整

完成後，至少確認壓縮檔中有：

- 學號命名的 HW2 Python 主程式
- 紅酒品質 CSV 資料集
- 完整 CRISP-DM PDF 報告
- NotebookLM 100 字以上研究摘要（可放在報告內）
- ChatGPT 對話紀錄 PDF
- 模型評估結果與相關圖片

你現在先做第一步：把我們的聊天紀錄存成 PDF，放到 `1015` 資料夾。

接著執行 `python prepare_submission.py`。如果出現錯誤或完成訊息，截圖給我，我幫你確認是否已經可以正式繳交。

你說：

我不能放跟agent聊天的紀錄嗎

ChatGPT 說：

可以！你可以把跟 Antigravity Agent 的聊天紀錄也放進作業裡。 而且這些對話能呈現你如何使用 AI 完成 CRISP-DM 分析、修改程式和解決錯誤。

不過，老師的原文是：

> 所有與 ChatGPT 的對話請以 pdfCrowd 或其他方式匯出為 PDF。

所以需要區分：

- **ChatGPT 對話紀錄**：老師明確要求，建議一定附上。
- **Antigravity Agent 對話紀錄**：可以額外附上，證明你使用 AI Agent 開發程式，但**無法確認老師是否接受它取代 ChatGPT 紀錄**。

我建議你兩份都放

你的作業資料夾可以整理成：

```
1015/
├── 7115056206_ChatGPT_Conversation.pdf
├── 7115056206_Antigravity_Conversation.pdf
├── 7115056206_hw2.py
├── HW2_Report.pdf
├── winequality-red.csv
└── prepare_submission.py
```

Antigravity Agent 對話怎麼匯出？

你可以先在 Agent 的對話介面查看 `...` 選單是否有匯出功能。如果沒有，可以嘗試透過畫面列印或複製對話內容整理成 PDF。

目前我無法確認你使用的 Antigravity 版本是否支援完整對話匯出。

我的建議：兩份都附上最保險。 如果你希望以 Agent 對話為主，可以把它當作「AI Agent 輔助開發紀錄」，而 ChatGPT 對話則作為老師指定的附件。

另外，打包前要確認 `prepare_submission.py` 會把兩份 PDF 都納入 ZIP，否則即使放在資料夾裡，也不一定會被打包進去。

你說：

我要怎麼把這裡的截圖 這裡那麼多

ChatGPT 說：

不用一張一張截圖！你跟我在 ChatGPT 的對話這麼長，**最方便的方法是直接匯出整段聊天紀錄，再轉成 PDF**。

老師要求的是對話紀錄 PDF，不是每一則訊息都要截圖。

方法一：直接把整段對話列印成 PDF（先試這個）

1. 用電腦瀏覽器打開我們現在這個 ChatGPT 對話。
2. 往上捲動，讓較早的對話載入。
3. 按鍵盤 **Ctrl + P**。
4. 在「目的地」選擇 **另存為 PDF**。
5. 檢查預覽有沒有包含前面所有對話。
6. 按「儲存」。

建議檔名：

```
7115056206_ChatGPT_Conversation.pdf
```

注意：ChatGPT 對話很長時，瀏覽器列印不一定會包含所有訊息，因此一定要檢查 PDF 的開頭與最後幾頁。

方法二：使用 ChatGPT 資料匯出功能

如果第一種方法無法匯出完整內容，可以使用 ChatGPT 的資料匯出功能。

1. 點 ChatGPT 左下角的個人頭像。
2. 進入 **Settings（設定）**。
3. 選擇 **Data Controls（資料控制）**。
4. 找到 **Export Data（匯出資料）**。
5. 依照畫面申請匯出。

這通常是匯出帳號的聊天資料，不是只匯出這一則對話。取得資料後，還需要把這次 HW2 的相關對話整理成 PDF。

我比較建議你怎麼做？

先使用 **方法一 Ctrl + P**，因為最快，也最符合老師要求的 PDF 格式。

如果列印出來有幾十頁甚至上百頁，不一定有問題，重點是對話內容要完整、可閱讀。

你現在先按 Ctrl + P，然後截圖給我看列印預覽畫面。 我可以幫你判斷這樣存下來會不會漏掉前面的對話。